

Received 12 May 2025, accepted 27 June 2025, date of publication 7 July 2025, date of current version 18 July 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3586673

TOPICAL REVIEW

A Benchmark Review of YOLO Algorithm Developments for Object Detection

ZHENGMAO HUA^{1,*}, KAVIYA ARANGANADIN^{1,*}, (Member, IEEE), CHENG-CHENG YEH^{2,3},
XINHE HAI¹, CHEN-YUN HUANG², TSAN-CHUEN LEUNG⁴, HUA-YI HSU⁵,
YUNG-CHIANG LAN⁵, AND MING-CHIEH LIN¹, (Senior Member, IEEE)

¹Multidisciplinary Computational Laboratory, Department of Electrical and Biomedical Engineering, Hanyang University, Seoul 04763, South Korea

²Lifetree Telemed Inc., Taichung 402256, Taiwan

³Department of Photonics, National Cheng Kung University, Tainan 70101, Taiwan

⁴Department of Physics, National Chung Cheng University, Chiayi 62101, Taiwan

⁵Department of Mechanical Engineering, National Taipei University of Technology, Taipei 10608, Taiwan

Corresponding author: Ming-Chieh Lin (mclin@hanyang.ac.kr)

*Zhengmao Hua and Kaviya Aranganadin are co-first authors.

ABSTRACT You Only Look Once (YOLO) has established itself as a prominent object detection framework due to its excellent balance between speed and accuracy. This article provides a thorough review of the YOLO series, from YOLOv1 to YOLOv10, including YOLOX, emphasizing their architectural advancements, loss function improvements, and performance enhancements. We have benchmarked the officially released versions from YOLOv3 to YOLOv10 and YOLOX, using widely recognized datasets VOC07+12 and COCO2017, on diverse hardware platforms: NVIDIA GTX Titan X, RTX 3060, and Tesla V100. The benchmark provides significant insights, such as YOLOv9-E achieving the highest mean average precision (mAP) of 76.0% on VOC07+12 and also showing superior detection accuracy on COCO2017 with an mAP of 56.6% which is 1.2% higher than that of the latest YOLOv10-X. YOLOv9-E stands out for its superior detection accuracy making it more suitable for detection that needs high accuracy such as analysis of medical images, while some lightweight versions like YOLOv5-S, YOLOv7-S, YOLOv8-S, and YOLOv10-S offer the great balance of accuracy and speed, making them ideal for real-time applications. Among them, YOLOv7-S has the highest mAP value among these lightweight models. Inference benchmarks highlight lightweight YOLO models such as YOLOv10-S for their exceptional inference speed on all GPUs and results of training time also indicate YOLOv9-E would take the longest time to converge among all versions using both datasets. This study would provide researchers and developers with some strategies in choosing appropriate YOLO models based on accuracy, resource availability, and application-specific needs.

INDEX TERMS COCO2017, computer vision, object detection, mAP, VOC07+12, YOLO.

I. INTRODUCTION

The birth of computer vision [1] was inspired by the visual systems of animals. Vertebrates, with their ability to effectively recognize and locate objects in their surroundings through complex visual systems, provided the foundation for researchers to develop computer vision technology. Since its inception in the mid-20th century, the field of computer vision had witnessed remarkable progress. Early research in this area focused on fundamental aspects such as image

processing and pattern recognition. A notable milestone was achieved in 1957, when Rosenblatt introduced the perceptron [2], laying the groundwork for artificial neural network (ANN)-based methodologies [3]. Over subsequent decades, advancements in algorithms such as edge detection [4], shape recognition [5], and motion tracking [6] further refined visual data analysis techniques. The 1980s and 1990s saw the development and adoption of feature extraction [7] and scene understanding [8] techniques, which later served as the foundation for more advanced methods. In the 21st century, machine learning techniques [9] especially deep learning [10], revolutionized the field. Algorithms like

The associate editor coordinating the review of this manuscript and approving it for publication was Zhenhua Guo¹.

support vector machines (SVMs) [11], random decision forests [12], and convolutional neural networks (CNNs) [13] enabled significant breakthroughs in tasks such as image classification [14], object detection [15], and semantic segmentation [16]. Among these tasks, object detection stands out as a critical component that not only identifies object categories but also determines their locations and sizes using bounding boxes, as illustrated in Fig. 1. Object detection is foundational to computer vision and has diverse practical applications [17], including autonomous vehicles [18], safety systems [19], medical imaging [20], and remote sensing [21].

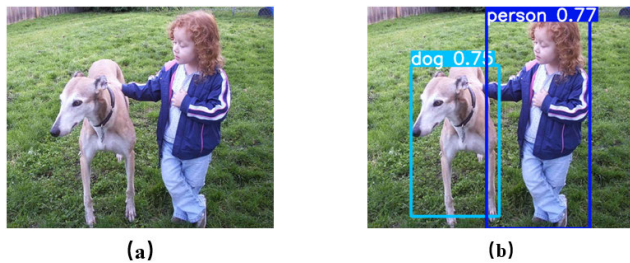


FIGURE 1. (a) The original image used for the object detection task and (b) the output result after the algorithm's processing, showing rectangular bounding boxes successfully detecting a dog and a person with 0.75 and 0.77 confidence values, respectively. The image is taken from the VOC07+12 dataset [29].

Early object detection techniques predominantly relied on traditional digital image processing methodologies, which served as the primary means for image classification and object localization. These methods typically followed a three-step process, as shown in Fig. 2. First, candidate regions were extracted from images to determine object positions, often using exhaustive sliding window [22] strategies to ensure comprehensive coverage despite the variability in object sizes and aspect ratios. Second, features were extracted from candidate regions using algorithms like scale-invariant feature transform (SIFT) [23] and histogram of oriented gradients (HOG) [24]. Finally, classifiers such as SVMs or Adaboost [25] were employed to classify features and iteratively train robust models. While these methods provided a baseline for object detection, they were limited in accuracy and efficiency, especially in complex and variable real-world scenarios. The computational cost of exhaustive region proposals (RPs) and feature extraction posed significant challenges, underscoring the need for more advanced and efficient approaches. The early 2010s marked a turning point

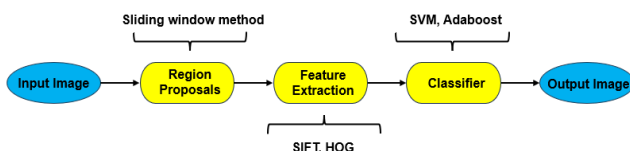


FIGURE 2. Representative algorithms used in various steps from input to output using traditional object detection methods.

in artificial intelligence (AI) with the rise of deep learning techniques, particularly CNNs, which revolutionized object detection in computer vision. Fig. 3 illustrates representative

object detection algorithms based on deep learning. Inspired by the structure and function of biological neural networks in a human brain, ANNs emulate interconnected neurons divided into input, hidden, and output layers, as shown in Fig. 4. The input layer receives input data and passes it to subsequent neurons. The hidden layer is located between the input and output layers and is used to process and abstract the input data. The output layer produces the final output. The ANN model shown in the figure is composed of n input values, m hidden units, and k output units. The direction arrow represents the direction in which the input values are transmitted. CNNs, a specialized form of ANNs, are particularly suited for tasks involving structured data like images, making them ideal for object detection. A CNN typically consists of convolutional layers, pooling layers, and fully connected layers. Convolutional layers extract spatial features by convolving input images with trainable filters, while pooling layers reduce spatial dimensions to enhance translation invariance. Fully connected layers, positioned at the network's end, perform classification or regression tasks based on the features extracted earlier.

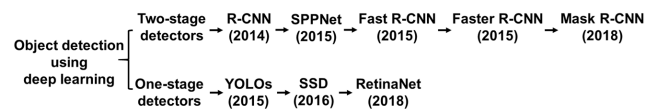


FIGURE 3. The development of deep learning object detection techniques. Following the integration of deep learning, there are two primary approaches for object detection detectors: one-stage and two-stage detectors.

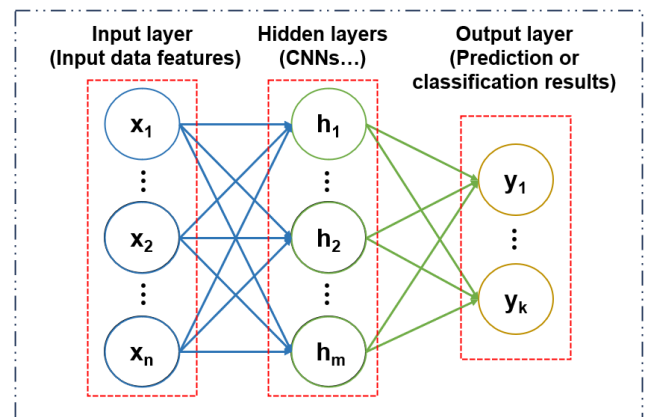


FIGURE 4. The structure of ANNs. ANN mainly consists of three layers with different functions.

Table 1 illustrates the differences between one-stage and two-stage object detection algorithms by comparing their backbone networks, datasets, detection speeds, and performance metrics. This overview serves as a foundation for the detailed discussion that follows. In 2014, Girshick et al. introduced deep learning into object detection with the region-based convolutional neural network (R-CNN) algorithm [26], which laid the foundation for deep learning-based object detection. R-CNN is a two-stage

TABLE 1. Comparative overview of one-stage and two-stage deep learning object detection algorithms, highlighting differences in method, backbone network, dataset utilized, detection speed in frames per second (FPS), mAP, and GPU hardware employed.

Category	Method	Backbone Network	Dataset	FPS	mAP	GPU
Two-stage detector	R-CNN [26]	AlexNet [101]	VOC2007	0.1	58.5	TitanX
	SPPNet [27]	ZFNet [107]	VOC2007	2.6	60.9	Titan X
	Fast R-CNN [28]	VGGNet [33]	VOC07+12	0.5	70	Titan X
	Faster R-CNN [30]	VGGNet	VOC07+12	7	73.2	Titan X
	Mask R-CNN [31]	ResNet [32]+FPN [38]	fine+COCO	195	41.7	NVIDIA Tesla M40
One-stage detector	YOLOv1 [34]	VGGNet19	VOC07+12	21	66.4	Titan X
	SSD [35]	VGGNet16	VOC07+12	46	74.3	Titan X
	RetinaNet [37]	ResNet+FPN	COCO	122	38.2	NVIDIA Tesla M40

detector that first generates RP through methods like selective search, which rely on predefined rules and patterns, and then classifies each proposal and performs bounding box regression in a separate stage. Although this approach significantly improved detection accuracy, it was computationally expensive, with long training times and high memory requirements. To address these limitations, He et al. proposed spatial pyramid pooling (SPP) in 2015 [27]. SPP network (SPPNet), also a two-stage detector, leveraged spatial pyramid pooling to handle inputs of varying sizes, enabling efficient feature extraction from multiple RPs with a single operation. This innovation reduced training time and enhanced flexibility for object detection tasks. Building on this, Girshick developed Fast R-CNN [28], which improved computational efficiency by integrating classification and bounding box regression directly into the network. As a two-stage detector, Fast R-CNN replaced the SVM classifier and bounding box linear regression of R-CNN with a unified architecture and introduced a region of interest (ROI) pooling layer to standardize RP sizes. Compared to R-CNN, Fast R-CNN achieved a 9x speed improvement during training and a 213x speed boost during testing, with a mean average precision (mAP) of approximately 66% on the PASCAL visual object classes 2007 (VOC2007) dataset [29]. In 2016, Ren et al. proposed Faster R-CNN [30], a two-stage detection technique based on R-CNN and Fast R-CNN models, which further streamlined the process by introducing a region proposal network (RPN). The RPN efficiently generated high-quality RPs using feature maps from pre-trained CNNs, enabling Faster R-CNN to integrate feature extraction, proposal generation, bounding box regression, and classification into a single network. This significantly improved both detection speed and accuracy. In 2018, He et al. extended Faster R-CNN to create Mask R-CNN [31], a sophisticated two-stage detection model that added a branch for segmentation mask prediction. Mask R-CNN improved registration accuracy by replacing the ROI pooling layer with an ROI alignment layer, reducing errors between input features and target objects. It also introduced pixel-level segmentation, enabling applications such as instance segmentation, object detection, and human

keypoint detection. Mask R-CNN uses residual networks (ResNet) [32] with 50 layers (ResNet-50) and 101 layers (ResNet-101) as the backbone for feature extraction, unlike Faster R-CNN, which relies on the visual geometry group (VGG) [33] network. Despite its high computational cost, Mask R-CNN has become a leading tool in applications like facial recognition, vehicle detection, and remote sensing object recognition. While two-stage object detection methods like Mask R-CNN offer high accuracy, their computational requirements and longer processing times highlight the need for further advancements in efficiency to address real-time constraints in practical applications.

On the other hand, Redmon et al. introduced You Only Look Once (YOLO) [34] in 2016, a groundbreaking one-stage object detection algorithm. YOLO eliminated the need for a candidate RP stage, integrating feature extraction, classification, and bounding box regression into a single branchless deep convolutional network. This simplification increased detection speed dramatically, from 7 frames per second (FPS) in Faster R-CNN to 45 FPS in YOLOv1. YOLOv1 became the first one-stage object detection algorithm to bypass the middle candidate region determination process, enabling a single network to simultaneously predict object categories and their bounding box locations. In 2016, Liu et al. proposed the single shot detector (SSD) for multiple categories [35], another noteworthy one-stage object detection technique. SSD can be seen as a hybrid of Faster R-CNN and YOLO, combining the regression-based approach of YOLO with region-based concepts from Faster R-CNN. Unlike YOLO, SSD addressed the challenges of detecting small objects by using feature maps of varying scales. Smaller-scale feature maps were employed for detecting larger objects, while larger-scale feature maps were used for inferring smaller objects. Additionally, SSD utilized anchors of various sizes and aspect ratios, overcoming YOLO's limitations in detecting and accurately localizing small objects. The SSD architecture was based on the VGG16 model, with additional convolutional layers added to extract more feature maps for object detection. On the VOC and the Microsoft Common Objects in Context (COCO) [36] datasets, SSD outperformed Faster R-CNN in both accuracy and speed, running at 59 FPS

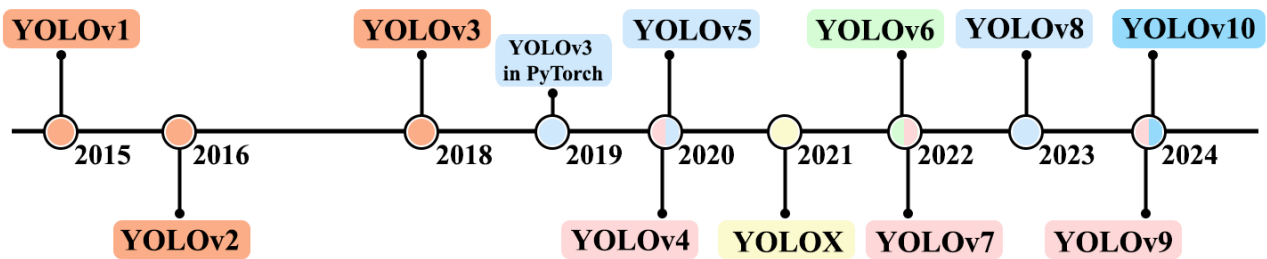


FIGURE 5. The evolution timeline of YOLO versions. The YOLO versions developed by the same team are highlighted with the same color.

in real-time faster than YOLOv1 while delivering improved detection performance. The RetinaNet [37], developed by Lin et al. in 2018, introduced a significant innovation in one-stage object detection. While RetinaNet's network structure was not revolutionary, it achieved a milestone by surpassing two-stage object detection algorithms in accuracy. The model combined a feature pyramid network (FPN) [38] and a fully convolutional network (FCN) to improve feature extraction across scales. However, RetinaNet's major breakthrough was the introduction of focal loss, a refined cross-entropy (CE) loss function designed to address the class imbalance between foreground and background samples in object detection tasks. By down-weighting the loss contribution from easy-to-detect objects, focal loss allowed the network to focus more on hard-to-detect objects, improving overall performance. Prior methods such as bootstrapping or hard example mining had attempted to address this imbalance, but focal loss provided a more effective and efficient solution. Experimental results demonstrated that RetinaNet achieved accuracy comparable to, or better than, two-stage detection algorithms, while maintaining the characteristic of faster inference speeds of one-stage models.

Although RetinaNet and other one-stage detection algorithms have improved accuracy and speed, the YOLO series remains the most popular choice for object detection in the industry. Its success is attributed to increased speed and accuracy, open-source availability, and continuous iteration over versions. The evolutionary timeline of the YOLO series is depicted in Fig. 5. The original YOLOv1 model, developed by Redmon et al. at the University of Washington, laid the foundation for this series. Redmon and his team continued refining the model until YOLOv3 [39] which was released in 2018, and the YOLO series had continued to evolve, with contributions from various teams. In early 2024, YOLOv9 [40] was launched, followed shortly by the recent version YOLOv10 [41], developed by a team from Tsinghua University.

The rapid iterations and widespread use of YOLO models have made them a standard in the industry. However, the increasing number of YOLO versions presents a challenge for users in selecting the most suitable model. Training deep models such as YOLO requires significant computational resources, and selecting an appropriate graphics processing unit (GPU) is crucial to ensure efficient training within a

reasonable timeframe while minimizing costs. Reviewing YOLO benchmarks and exploring GPU configurations across various performance levels are essential to guide users in making informed decisions about model selection and resource allocation. Despite YOLO's popularity, the literature lacks comprehensive reviews dedicated to its algorithms, particularly benchmarks based on mainstream datasets and GPU configurations. Jiang et al. [42] reviewed the evolution of YOLOv1 to YOLOv5 and the feature extraction methods employed in these versions. They also analyzed publishing trends from 2016 to 2020, providing insights into YOLO's developmental trajectory. However, their study did not cover versions beyond YOLOv5, nor did it address algorithm performance analysis, dataset discussions, or benchmark testing. Zhao et al. [43] introduced convolutional neural networks as a tool for deep learning, and summarized object detection models based on deep learning, including YOLOv1 and YOLOv2. However, their paper primarily focused on these foundational models, limiting its scope in analyzing the broader YOLO algorithm. Sirisha et al. [44] provided a more extensive review covering YOLOv1 to YOLOv7, including their variants and mainstream datasets used for training. Nevertheless, their work overlooked YOLOv8 and related benchmark testing. Terven et al. [45] offered a detailed overview of YOLOv1 to YOLOv8 and their variants but did not address the datasets and application domains of YOLO models. Hussain's 2023 review [46] focused primarily on the industrial applications of YOLO algorithms, providing practical use cases but lacking examples from other fields. In contrast, his 2024 review [47] expanded the scope by exploring advancements in YOLOv1 to YOLOv8, adding applications in non-industrial fields, and analyzing challenges and future development directions for the YOLO series. However, neither of these studies, to date, has discussed YOLOv9 or YOLOv10, leaving a gap in the literature regarding the latest advancements in the YOLO series.

The YOLO family of object detection algorithms has continuously evolved to meet the growing demand for real-time and high-accuracy vision applications. Although many versions of YOLO have been developed, a unified benchmark study that evaluates the officially released YOLO models under consistent experimental conditions has been largely missing in the literature. This article addresses that gap by focusing exclusively on official versions, including

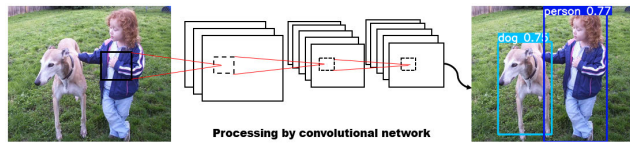


FIGURE 6. The implementation process of YOLO algorithm. Firstly, adjust the size of the input image, then run the convolutional neural network, and finally output the processed predicted box and confidence level.

YOLOv3 through YOLOv10, as well as YOLOX. Modified or unofficial YOLO variants developed by independent research groups are not considered, in order to ensure consistency in evaluation and comparison. All benchmark experiments are conducted using three widely adopted GPU platforms: NVIDIA GTX Titan X, RTX 3060, and Tesla V100. These platforms represent a range of hardware configurations commonly used by researchers and practitioners. For the benchmark, we selected the small (S) and extra large (X) variants from YOLO versions that provide them, as these configurations represent the lower and upper bounds in terms of model size, computational complexity, and detection accuracy. Based on these two representative configurations, researchers can infer expected trade-offs of intermediate variants, depending on their specific application requirements and resource availability. The benchmark is performed using two widely recognized datasets, VOC07+12 and COCO2017. The evaluation covers key performance indicators, including mAP, inference speed, and training convergence time. Most importantly, to our knowledge, this is the first article to benchmark multiple official YOLO versions on mainstream datasets using GPUs with a range of configurations, from low to high.

The remainder of this article is structured as follows. Section II provides an overview of the official YOLO algorithm series, detailing architectural improvements, loss functions, and key contributions from YOLOv1 to YOLOv10, including YOLOX. Section III outlines common application domains of YOLO-based object detection systems across various industries. Section IV describes the benchmarking methodology, including the datasets used, evaluation metrics, and hardware configurations. Section V presents the benchmark results, highlighting trends in accuracy, training time, and inference speed across GPU platforms. Finally, Section VI concludes the article with a summary of findings. This article aims to provide valuable insights for researchers and practitioners integrating YOLO into their applications while guiding future research and development in the field.

II. OVERVIEW OF THE YOLO ALGORITHM SERIES

A. ARCHITECTURAL DESIGN AND KEY COMPONENTS OF YOLO ALGORITHM

The YOLO algorithm revolutionized object detection by adopting a unique, innovative approach. Unlike Faster R-CNN networks, which adopt region-based predictions, YOLO directly uses the entire image as input and performs

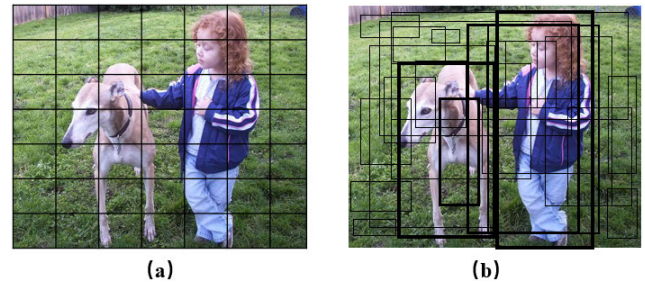


FIGURE 7. (a) Process of dividing an image into uniform grid cells. (b) A scenario where each grid cell generates multiple bounding boxes, covering potential object locations.

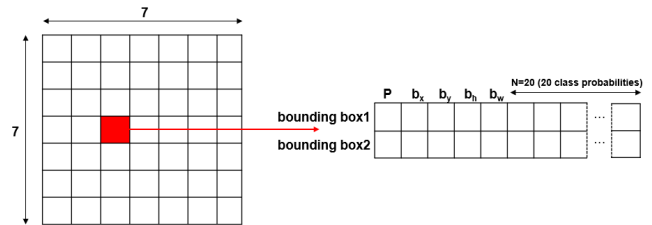


FIGURE 8. The process of generating bounding boxes for one grid cell. Each grid cell generates two bounding boxes, and each bounding box is characterized by five parameters: the center position denoted by (b_x and b_y), its height (b_h), its width (b_w), and the probability (P) indicating the likelihood of the bounding box containing the predicted object.

simultaneous object category prediction and bounding box regression in a single pass through the network. This architectural design combines object localization and classification in a unified model, significantly enhancing detection speed and efficiency. As illustrated in Fig. 6, the three main components of YOLO's design are the input layer, feature extraction layer, and output layer, designed to locate objects accurately and identify their categories and positions.

1) GRID-BASED IMAGE SEGMENTATION AND BOUNDING BOX PREDICTION

As shown in Fig. 7, YOLO begins by dividing the input image into a grid of uniformly sized cells. Unlike the sliding window methods used in traditional approaches like R-CNN, YOLO's grid-based strategy allows multiple objects to be detected in a single frame. A key innovation is that the grid cell is responsible for predicting an object only if the center of the object's bounding box falls within it. This eliminates redundant computations and ensures efficient handling of overlapping objects. As illustrated in Fig. 8, YOLO divides the input image into a grid of size $S \times S$ (e.g., 7×7), with each grid cell responsible for detecting objects whose center falls within that cell. Each grid cell predicts B bounding boxes. Here, each bounding box includes five parameters: the center coordinates (b_x , b_y), width (b_w), height (b_h), and the probability (P), which represents the likelihood that the box contains an object. Additionally, the grid cell predicts N class probabilities, corresponding to the possible object categories. Altogether, each grid cell predicts $B \times 5 + N$ values. For the entire grid, the total number of predicted values is $S \times S \times (B \times 5 + N)$. For example, in a 7×7 grid with $B = 2$ bounding boxes per cell and $N = 20$ object

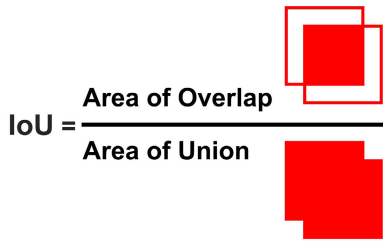


FIGURE 9. IoU calculates the overlap between the ground truth bounding box an object in an image and the predicted bounding box (or region).

classes, each grid cell predicts 30 values, resulting in a total of $7 \times 7 \times 30 = 1,470$ predictions for the whole image. These predictions form the output tensor, structured as $S \times S \times (B \times 5 + N)$. Fig. 7(b) depicts the scenario when all bounding boxes are generated by grid cells. The confidence score for each bounding box is computed using the following formula:

$$C = \text{Pr}(\text{obj}) * \text{IoU}_{\text{truth}}^{\text{pred}}, \quad (1)$$

where, $\text{IoU}_{\text{truth}}^{\text{pred}}$ represents the intersection over union (IoU) between the predicted bounding box and the ground truth bounding box, as shown in Fig. 9, and $\text{Pr}(\text{obj})$ is the probability that a grid cell contains an object, where this probability is 1 if the grid cell contains an object and 0 otherwise.

2) HANDLING OVERLAPPING DETECTIONS AND NON-MAXIMUM SUPPRESSION

As shown in Fig. 10, overlapping bounding boxes are a common occurrence in object detection, especially when large objects span multiple grid cells. To address this, YOLO applies a class confidence score to each bounding box to quantify the likelihood of an object's presence. A confidence threshold is set to eliminate low-confidence bounding boxes, significantly reducing the total number of candidate boxes. However, even after applying the threshold, overlapping bounding boxes may remain. To resolve this, YOLO uses non-maximum suppression (NMS) [48]. The steps involved in NMS are: (i) identify the bounding box with the highest confidence score and designate it as the “maximum bounding box”, (ii) calculate the IoU between the maximum bounding box and all other boxes, (iii) remove all bounding boxes with an IoU greater than a predefined threshold, because these bounding boxes are deemed to predict the same object, and (iv) repeat the process until only non-overlapping, highest-confidence bounding boxes remain. This ensures that each object is detected with least redundancy and best accuracy. The final output combines the grid cell information with the type and position of the highest confidence bounding box, achieving precise object recognition and localization.

B. DEVELOPMENT OF YOLO VERSIONS

1) YOLOv1

YOLOv1 was introduced by Redmon et al. in their 2016 paper [34], marking a significant milestone in object detection and

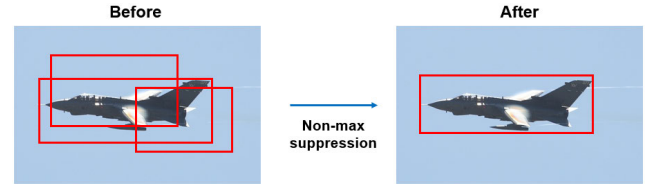


FIGURE 10. Implementation of non-maximum suppression (NMS) algorithm. NMS keeps only the best bounding boxes and removes overlapping ones to avoid redundancy in object detection. This image is taken from the VOC2007 dataset [29].

computer vision research. YOLOv1 uses a backbone network architecture similar to GoogleNet [49], as illustrated in Fig. 11, and comprises 24 convolutional layers and two fully connected layers. The large square in the figure is the feature map, and the small square inside is the convolution kernel. The corresponding numbers are their length, width, and channel size. The description of each pair of feature maps in the figure outlines the convolutional layers and max-pooling layers applied within these maps. The notations specify the number, size, and configuration of the layers. For example, “s-2” indicates that the stride size is 2. Some feature maps share identical convolutional layers. For instance, the first two convolutional layers in the fourth and fifth feature maps are the same, each consisting of four layers. The input image is first resized to 448×448 pixels before being passed into the network, generating a $7 \times 7 \times 30$ output tensor. This tensor is divided into a 7×7 grid, where each grid cell predicts two bounding boxes, a confidence score for each box, and the class probabilities of detected objects. Thus, YOLOv1 can detect up to $7 \times 7 \times 2 = 98$ bounding boxes and identify a maximum of 49 objects in an image. However, its ability to detect small or densely packed objects is limited.

The performance of YOLOv1 is guided by its loss function, which quantifies the difference between predicted values and ground truths during training, which involves adjusting the periodic weight reduction through optimization algorithms such as random gradients. Hence, the loss function balances localization, confidence, and classification errors to guide the network toward accurate predictions. Its formula is as follows:

Loss

$$\begin{aligned} &= \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\ &+ \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{\text{obj}} \left[(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right] \\ &+ \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\ &+ \sum_{i=0}^{S^2} 1_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2, \end{aligned} \quad (2)$$

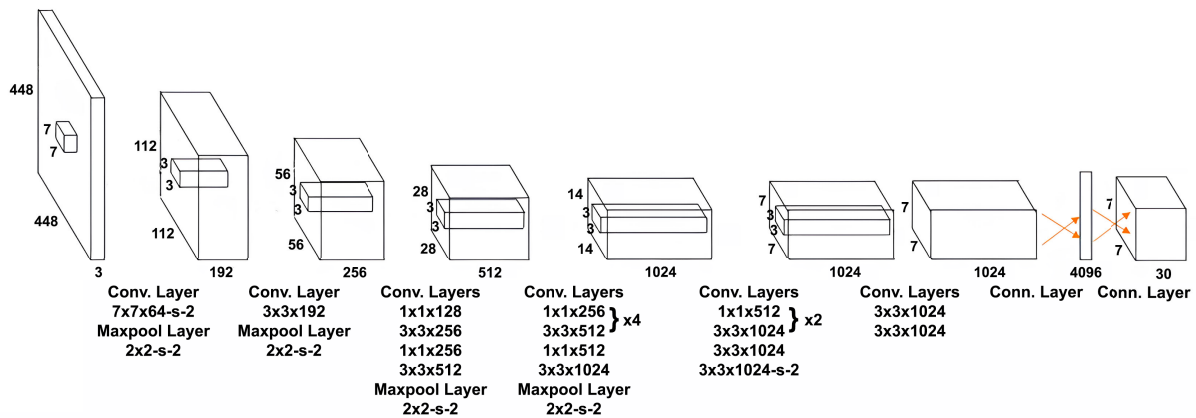


FIGURE 11. Model network structure of YOLOv1 [34]. The square represents the size of the input image. The input to the network is an image resized to 448×448 pixels with red, green, blue (RGB) color channels. After being processed by the convolution layer, the maximum pooling layer and the fully connected layer, the final output is $7 \times 7 \times 30$.

where, λ_{coord} and λ_{noobj} are hyperparameters that balance localization and confidence losses, respectively. 1_{ij}^{obj} is an indicator function that equals 1 if the j^{th} bounding box in the i^{th} grid cell is responsible for predicting an object or 0 otherwise. 1_{ij}^{noobj} equals 1 if the bounding box does not predict any object. x_i , y_i , w_i , and h_i are the predicted coordinates, width, and height of the bounding box, while $\hat{x}_i$, $\hat{y}_i$, $\hat{w}_i$, and $\hat{h}_i$ are the ground truth values. The $p_i(c)$ and $\hat{p}_i(c)$ are the predicted and true probabilities for class c in the i^{th} grid cell, respectively. The C_i and $\hat{C}_i$ are the predicted confidence and the ground truth confidence, respectively. The loss function in YOLOv1 is divided into five key components, each addressing a specific aspect of object detection. The localization loss for center coordinates ensures that the predicted center of a bounding box aligns closely with the center of the ground truth bounding box, improving spatial accuracy. The localization loss for width and height includes a square root term, which emphasizes accuracy for larger bounding boxes while reducing sensitivity to minor deviations in smaller boxes. The confidence loss for predicted bounding boxes evaluates the model’s confidence in detecting objects and penalizes errors in confidence estimation. Additionally, the no-object confidence loss minimizes false positives by reducing confidence scores for grid cells without objects. Finally, the classification loss penalizes the model for incorrectly predicting object categories, encouraging accurate class predictions for detected objects. These components work together to guide the model during training, ensuring precise detection and classification of objects.

YOLOv1 is a significant milestone in computer vision due to its speed and simplicity. Its unified architecture allows for end-to-end training, enabling real-time image processing at remarkable speeds. This makes it ideal for applications such as robotics, surveillance, and self-driving cars. However, YOLOv1 has notable limitations, including a difficulty in detecting small objects and overlapping objects.

These limitations arise from the $S \times S$ grid structure, which restricts the ability to handle densely packed or small-scale features. Subsequent versions of YOLO addressed these challenges by improving network architecture and loss functions.

2) YOLOv2

In 2017, Redmon and Farhadi introduced YOLOv2 [50], an enhanced version of YOLOv1 that addressed many limitations of the original model while introducing new features for improved performance. YOLOv2 replaced YOLOv1’s VGG-based backbone with a new architecture called Darknet-19, as shown in Table 2. The model includes nineteen convolutional layers and five max pooling layers. The final output, after going through Darknet-19 processing, will have a completely connected layer that can create a 7×7 grid. Thirty values are assigned to each grid cell, representing ten sets of bounding box coordinates from twenty different categories. In order to normalize the network’s output to the distribution of probabilities over anticipated output classes, the softmax layer is frequently employed as the final activation function of a neural network. Unlike conventional classification models, YOLOv2 does not use the softmax function. So the last row in the table representing the softmax method is empty. In order to increase efficiency, various layers that are often utilized for downsampling (such as max-pooling) are eliminated. The model employs 3×3 convolutional layers for feature extraction and 1×1 convolutions to compress feature map channels, reducing computational costs and parameter space. A significant enhancement was the addition of batch normalization (BN) layers after each convolutional layer, which accelerated convergence and prevented overfitting. The BN algorithm has been widely adopted in deep learning since its introduction in 2015. The purpose of this technique is to maintain the same input distribution for each layer of a neural network during the training process, thereby enhancing its performance. The BN

process divides the input data into small batches for stochastic gradient descent. Normalization is performed on each layer when each batch of data is passed through the neural network. This normalization is achieved by adding a BN layer between the fully connected and activation layers. By removing dropout layers and incorporating BN, YOLOv2 achieved a 2% improvement in mAP without additional computation costs. Another key improvement in YOLOv2 was the use of a high-resolution classifier to address the limitations of pre-trained networks, such as VGG and ResNet, which were typically trained on low-resolution images (256×256). YOLOv2 fine-tuned its network on the ImageNet [51] dataset for 10 epochs using higher-resolution images of 448×448 , allowing it to adapt better to high-resolution inputs. This modification increased the model's mAP by approximately 4%, enhancing its ability to detect small objects. YOLOv2 also replaced YOLOv1's fully connected layers with anchor boxes, which eliminated the loss of spatial information and improved positioning accuracy. Additionally, one pooling layer was removed to produce higher-resolution feature maps, resulting in a 13×13 feature map for an input image size of 416×416 . Each grid cell in the feature map predicted five anchor boxes, including location information, confidence scores, and class probabilities. This adjustment increased the model's recall rate from 81% to 88%, although the mAP dropped slightly from 69.5% to 69.2%. The use of anchor boxes improved the detection of objects with varying scales but introduced a minor trade-off in precision, which decreased by 0.3%.

To enhance the bounding box predictions further, YOLOv2 introduced dimension clustering using the k-means clustering algorithm [52]. Instead of manually defining anchor boxes, the clustering algorithm optimized anchor box dimensions based on the IoU values of bounding boxes in the training dataset. This data-driven approach improved alignment between anchor boxes and object sizes, making the model easier to train and more accurate in detecting objects.

A unique feature of YOLOv2 was its ability to handle input images of varying resolutions through multi-scale training. The model randomly resized input images during training, enabling the network to adapt to multi-scale inputs. High-resolution inputs improved accuracy (mAP), while low-resolution inputs increased processing speed, offering a flexible trade-off between speed and performance. Lastly, YOLOv2 incorporated fine-grained features by leveraging the skip connection concept from ResNet. This involved reducing the resolution of the $26 \times 26 \times 64$ feature map into a $13 \times 13 \times 256$ feature map and concatenating it with the $13 \times 13 \times 1,024$ feature map. The resulting $13 \times 13 \times 1,280$ feature map provided richer information for predictions, improving object detection accuracy, particularly for small and overlapping objects. In summary, YOLOv2 introduced significant architectural changes and novel techniques, including Darknet-19, batch normalization, anchor boxes, dimension clustering, multi-scale training,

TABLE 2. The architecture of the YOLOv2 model, based on the Darknet-19 backbone [50]. This table outlines the sequence of convolutional, pooling, and activation layers, along with their respective parameters such as filter, size/stride, and output.

Type	Filters	Size/Stride	Output
Conv.	32	3×3	224×224
Maxpool		$2 \times 2 / 2$	112×112
Conv.	64	3×3	112×112
Maxpool		$2 \times 2 / 2$	56×56
Conv.	128	3×3	56×56
Conv.	64	1×1	56×56
Conv.	128	3×3	56×56
Maxpool		$2 \times 2 / 2$	28×28
Conv.	256	3×3	28×28
Conv.	128	1×1	28×28
Conv.	256	3×3	28×28
Maxpool		$2 \times 2 / 2$	14×14
Conv.	512	3×3	14×14
Conv.	256	1×1	14×14
Conv.	512	3×3	14×14
Conv.	256	1×1	14×14
Conv.	512	3×3	14×14
Maxpool		$2 \times 2 / 2$	7×7
Conv.	1024	3×3	7×7
Conv.	512	1×1	7×7
Conv.	1024	3×3	7×7
Conv.	512	1×1	7×7
Conv.	1024	3×3	7×7
Conv.	1000	1×1	7×7
Avgpool		Global	1000
Softmax			

and fine-grained features. These enhancements collectively improved YOLOv2's recall, adaptability, and training efficiency, establishing it as a substantial advancement over YOLOv1.

3) YOLOv3

YOLOv3 was a new generation YOLO object detection framework introduced by Redmon and Farhadi in 2018. Building on the strengths of YOLOv2, YOLOv3 addressed limitations in detecting small objects and introduced several key innovations to enhance accuracy and robustness. One of its most significant upgrades was the adoption of Darknet-53, a deeper and more powerful backbone network. As illustrated in Fig. 12, Darknet-53 consists of 52 convolutional layers

	Type	Filters	Size	Output
1 x	Conv.	32	3 x 3	256 x 256
	Conv.	64	3 x 3 / 2	128 x 128
	Conv.	32	3 x 3	128 x 128
	Conv.	64		
	Residual			
2 x	Conv.	128	3 x 3 / 2	64 x 64
	Conv.	64	1 x 1	64 x 64
	Conv.	128	3 x 3	
	Residual			
8 x	Conv.	256	3 x 3 / 2	32 x 32
	Conv.	128	1 x 1	32 x 32
	Conv.	256	3 x 3	
	Residual			
8 x	Conv.	512	3 x 3 / 2	16 x 16
	Conv.	256	1 x 1	16 x 16
	Conv.	512	3 x 3	
	Residual			
4 x	Conv.	1024	3 x 3 / 2	8 x 8
	Conv.	512	1 x 1	8 x 8
	Conv.	1024	3 x 3	
	Residual			
	Avgpool		Global	
	Conn.		1000	
	Softmax			

FIGURE 12. The network structure of Darknet-53.

and 1 fully connected layer organized with a combination of simple convolutional layers and residual blocks. The numbers of layers and residual blocks for each stage are given in the box. In the YOLOv3 network, the total number of convolutional layers and fully connected layer is 53, so the network is called Darknet-53. In addition, the total number of residual blocks in the network is 23. The Darknet-53 integrates residual blocks, inspired by ResNet, as depicted in Fig. 13. Here, $f(x)$ represents the learned residual function applied to the input x . It usually consists of a convolution operation, batch normalization, and rectified linear unit (ReLU) activation. Essentially, $f(x)$ is the output of a set of operations applied to the input x and x represents the input to a layer in the network. It can be the original input image or a feature map passed from a previous layer in the network. The residual block contains one or more convolutional layers and a shortcut connection, allowing the network to be trained more easily without losing information. These residual blocks allow the network to learn abstract and discriminative features without performance degradation, enabling the model to handle complex and cluttered scenes effectively. Residual

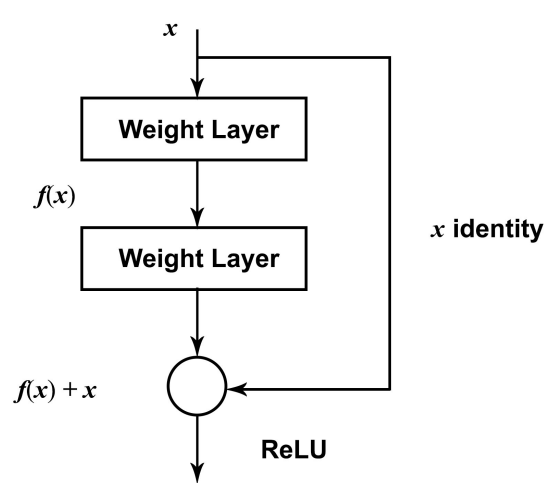


FIGURE 13. The structure of a residual block.

connections, first introduced in ResNet, play a crucial role in YOLOv3 by enabling the network to grow deeper while maintaining stability. This depth allows YOLOv3 to capture intricate patterns and variations in data, improving its ability to detect objects across diverse scenarios. The integration of residual blocks not only enhances feature extraction but also improves the overall training and inference performance of the model, making YOLOv3 a robust solution for real-world applications.

YOLOv3 also incorporates FPN, a technique introduced in 2017, to improve multi-scale feature extraction. As shown in Fig. 14, FPN integrates features from different layers of the network through the bottom-up and top-down processes. The bottom-up path represents the forward feature extraction of the convolutional layers, while the top-down path upsamples the feature maps from deeper layers. By combining high-level semantic information with low-level spatial details, YOLOv3 achieved superior performance in detecting both large and small objects within the same image. This multi-scale approach significantly enhances the detection of small objects, a common challenge in previous YOLO versions. To further optimize its architecture, YOLOv3 eliminated pooling and fully connected layers, replacing them with strides in convolutional layers to achieve tensor size transformations during forward propagation. This design choice minimizes information loss and maintains spatial details throughout the network. Additionally, YOLOv3 introduced support for predicting bounding boxes with varying aspect ratios, further improving its versatility in detecting objects of different shapes and sizes. With these enhancements, YOLOv3 represented a substantial leap forward in object detection technology. Its deeper architecture, residual connections, multi-scale feature fusion, and ability to predict bounding boxes with diverse aspect ratios make it a powerful and accurate next-generation object detection framework. These innovations ensure that YOLOv3 is well-suited for

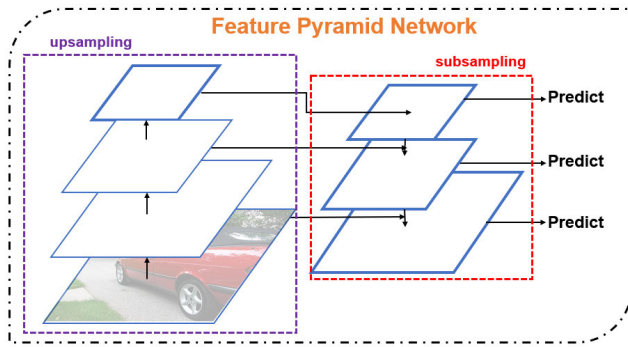


FIGURE 14. The structure of FPN. The FPN comprises two primary components. The example image is taken from the PASCAL VOC2007 dataset [29].

a wide range of applications, including scenarios involving small or densely packed objects.

4) YOLOv4

In 2020, Bochkovskiy et al. introduced YOLOv4 [53], a groundbreaking object detection algorithm that built upon YOLOv3 with significant improvements in architecture and methodology. At its core, YOLOv4 features the cross stage partial network (CSPDarknet-53) backbone, an enhanced version of the Darknet architecture launched in 2019 [54]. CSPDarknet-53 divides the network into smaller sub-networks, or stages, connected via cross-stage connections. This design reduces computational complexity and enhances feature extraction, making YOLOv4 deeper, more efficient, and more capable than its predecessors. Additionally, YOLOv4 employs larger pixel input sizes, which improves its ability to detect small objects. To address multi-scale object detection challenges, YOLOv4 integrates SPP, which reduces overfitting while preserving image scale, thereby enabling robust detection of objects across varying sizes. YOLOv4 introduced several key innovations that further enhance its performance, including Mosaic data augmentation, Mish activation [55], and the complete-IOU (CIoU) [56] loss function. Mosaic data augmentation, as illustrated in Fig. 15, combines four different images into a single training sample. This technique significantly diversifies the training dataset, enabling YOLOv4 to perform better in detecting small or densely packed objects. By increasing the variability in the input data, Mosaic augmentation improves generalization and reduces the risk of overfitting. The Mish activation function, defined as $y = x * \tanh[\ln(1 + e^x)]$, is another critical innovation in YOLOv4. As depicted in Fig. 16, Mish is a smooth, non-monotonic function with unbounded outputs, which prevents gradient saturation and accelerates training. Its behavior for $x > 0$ is similar to leaky rectified linear unit (Leaky ReLU) [57], while its smoothness for $x < 0$ enhances the model's generalization capabilities that allow the function to optimize the outcome effectively, improving detection quality. Although Mish has a higher computational cost than traditional activation functions like ReLU [58]



FIGURE 15. Mosaic technology. By randomly combining four pictures into a new picture, the diversity of the dataset is increased and the generalization ability of the model is improved. These images are taken from the VOC2007 dataset [29].

and Swish [59], its integration with the CSPDarknet-53 backbone significantly improves detection accuracy, making it a valuable addition to YOLOv4.

YOLOv4 also introduced the CIoU loss function, an advancement over the distance-IOU (DIOU) [60] to refine bounding box regression. DIOU considers three geometric parameters in bounding box regression: the overlapping area, the distance between center points, and the aspect ratio. CIoU extends this by adding a term that penalizes discrepancies in the scale (length and width) of the predicted bounding box relative to the ground truth. This refinement ensures tighter alignment with object boundaries and improves the model's overall detection accuracy. The formula for CIoU is:

$$\text{CIoU} = 1 - \text{IoU} + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v. \quad (3)$$

The CIoU loss function incorporates several key components to refine object detection performance. Euclidean distance (ρ) between the centers of the predicted (b) and ground truth boxes (b^{gt}) is another critical factor, ensuring that the predicted bounding box is positioned correctly relative to the target. To account for scale, the diagonal length (c) of the smallest enclosing box is used, penalizing predictions that deviate in size from the ground truth. Additionally, the loss function evaluates the aspect ratio consistency (v) between the predicted and ground truth boxes, ensuring that the shapes of the bounding boxes are well-matched. The weighting factor (α) further adjusts the importance of the aspect ratio term, providing flexibility in balancing various geometric aspects of the bounding box. Together, these components make CIoU a comprehensive and effective loss function for improving localization accuracy in object detection tasks. In summary, YOLOv4 incorporates numerous innovations that elevate its accuracy and robustness. By leveraging the CSPDarknet-53 backbone, SPP, Mosaic data augmentation, Mish activation, and CIoU loss, YOLOv4 delivers significant improvements over its predecessors. These advancements make it a powerful and efficient object detection framework, suitable for applications requiring high precision and scalability.

5) YOLOv5

YOLOv5 was introduced in 2020 by Ultralytics, a research company specializing in computer vision and machine

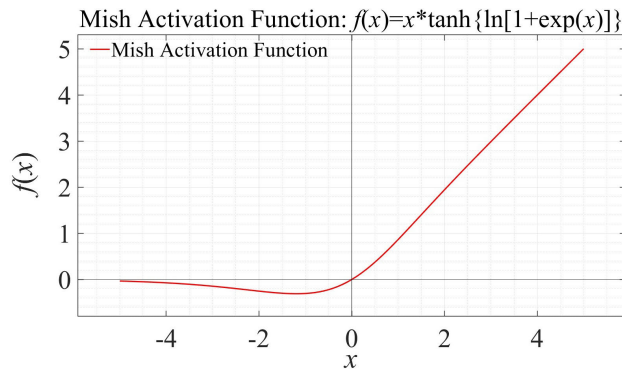


FIGURE 16. Mish activation function. The equation of Mish activation function is $f(x) = x * \tanh[\ln(1 + e^x)]$.

learning. Its network structure closely resembled YOLOv4 and continued to utilize Mosaic data augmentation at the input stage. During training, datasets often contain uneven distributions of small, medium, and large objects, with small objects typically achieving much lower average precision (AP) values. This imbalance can result in unsatisfactory overall AP values. The Mosaic augmentation technique, originally introduced in YOLOv4 and adopted by YOLOv5, addresses this issue by randomly scaling, cropping, and arranging four images into one composite image, as illustrated in Fig. 15. This technique enriches the dataset with diverse small object instances, making the model more robust in detecting objects across varying scales. YOLOv5 also incorporates adaptive anchor boxes to improve object detection across datasets with varying object sizes. Anchor boxes with preset lengths and widths are calculated adaptively based on the dataset. During training, the network generates predicted bounding boxes from these initial anchor boxes, compares them with the ground truth, calculates the difference, and updates the network parameters through back propagation. This process refines anchor box dimensions and improves detection performance for diverse datasets. A significant innovation in YOLOv5 is the introduction of the Focus [61] structure, designed to reduce computational complexity and increase training speed. The Focus structure in YOLOv5 is used to slice the image before it enters the backbone. As depicted in Fig. 17, this technique slices the picture and gets a value for every other pixel, forming four complementary and similar-sized pictures without losing information. Doing so can concentrate the width and height information into the channel space and increase the input channels by four times, from the original red, green, blue (RGB) three-channel mode to 12 channels. Finally, a convolution operation is performed on the new image to obtain a twice-downsampled feature map without information loss. The original $608 \times 608 \times 3$ input image is sliced and transformed into a $304 \times 304 \times 12$ feature map. This feature map undergoes convolution with 32 kernels, resulting in a $304 \times 304 \times 32$ processed output. By effectively handling spatial information early in the network, the Focus structure enhances efficiency without compromising accuracy. Another enhancement in

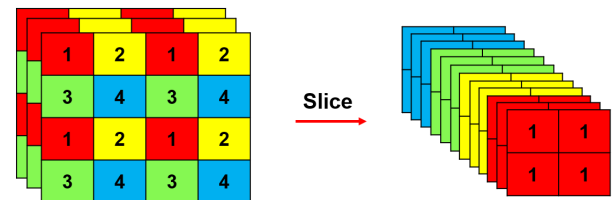


FIGURE 17. Slicing operations: slice the picture and get a value for every other pixel, forming four complementary and similar-sized pictures without losing information.

YOLOv5 is the adoption of two CSP structures, CSP1 and CSP2, which are utilized in the backbone and neck portions of the network, respectively. While YOLOv4 also utilizes CSPs, YOLOv5's implementation improves feature fusion capabilities. The CSP2 structure, specifically designed based on CSPNet, strengthens the network's ability to combine features from different layers, particularly in the neck part, which diverges from the standard convolutional operations used in YOLOv4. Additionally, the YOLOv5's open-source framework provides four pre-trained models with varying network structures. These models differ in depth and width, corresponding to the depth of the CSP structure. The larger models have more convolution kernels, resulting in thicker feature maps and greater feature extraction capabilities. This flexibility allows YOLOv5 to cater to projects with different computational and accuracy requirements. Users can select smaller models for speed-critical tasks or larger models for high-precision applications, making YOLOv5 versatile and adaptable for a wide range of use cases. In summary, YOLOv5 advances the YOLO series with its Mosaic data augmentation, adaptive anchor boxes, Focus structure, and improved CSP designs. By providing pre-trained models with varying configurations, YOLOv5 offers a flexible and robust solution for diverse object detection tasks, further solidifying its position as a state-of-the-art detection framework.

6) YOLOX

In 2021, the Megvii Research Institute introduced YOLOX [62], a groundbreaking extension of the YOLO object detection series. This innovative model incorporates state-of-the-art practices in object detection, such as decoupled head, anchor-free methods, enhanced data augmentation, and label classification, while building upon the YOLO framework. YOLOX not only surpasses the AP of its predecessors YOLOv3, YOLOv4, and YOLOv5, but also maintains competitive inference speeds. These advancements make YOLOX a versatile and powerful model for modern object detection tasks. Mixup was adopted in the context of object detection, including models like YOLOv3 and subsequent versions, to improve generalization and reduce overfitting by blending two training images and their labels. Similar to YOLOv5, to enhance the detection of small objects, YOLOX retains the input method from previous YOLO versions while significantly improving Mosaic and Mixup data augmentation techniques, inherited from the

YOLOv3 baseline. These augmentations diversify the training dataset and improve the model's robustness. Unlike traditional anchor-based methods, YOLOX adopts an anchor-free approach, eliminating the dependence on preset anchor sizes, numbers, and aspect ratios. In anchor-based systems, such as earlier YOLO models, the network is sensitive to anchor configurations, which require careful tuning. Fixed anchor sizes and aspect ratios make it challenging to detect objects with significant scale variations, such as tiny objects, limiting the model's generalization ability. YOLOX resolves this issue by replacing anchors with a center-based mechanism.

In YOLOX's anchor-free method, the center point of the predicted bounding box is defined as the origin, and the four vertices of the bounding box are determined relative to this origin. Inspired by the fully convolutional one stage (FCOS) network, this approach removes the need for time-consuming and memory-intensive anchor-related computations, such as intersection and comparison operations. Instead, each position on the output feature map directly predicts four numerical values to determine the offsets for the top-left and bottom-right corners of the bounding box. This simplification enhances the model's flexibility, efficiency, and generalization ability. Another significant innovation in YOLOX is its decoupled head structure in the output layer, which differs from the coupled detection heads used in earlier YOLO models. The decoupled head processes classification and regression tasks separately, dividing the features into two parallel streams using bidirectional convolution. A dimensionality reduction step reduces the number of parameters, improving efficiency. This separation allows the model to independently learn features needed for classification and regression, simplifying the learning process and improving overall performance. This structural change also accelerates the network's convergence during training and leads to superior results on multiple benchmark datasets, including the COCO and the VOC datasets. In addition to surpassing previous YOLO models in mAP scores, YOLOX outperformed other leading object detection frameworks such as EfficientDet [63] and DETection TRansformer (DETR) [64]. Furthermore, YOLOX achieved faster inference times compared to earlier YOLO versions, making it an ideal choice for real-time applications. With its combination of innovative techniques and efficient architecture, YOLOX represents a major advancement in object detection, offering both high precision and speed for modern detection tasks.

7) YOLOv6

In 2022, Meituan's Visual Intelligence Department developed YOLOv6 [65], a next-generation object detection model tailored for industrial applications. Building upon the strengths of YOLOv5, YOLOv6 introduced numerous enhancements and features to improve performance and efficiency. At its core, YOLOv6 uses the EfficientRep backbone, a variant of the RepVGG network, which employs a multi-stage convolutional layer system. This backbone progressively

downsamples the input image to capture information at different scales, optimizing feature representation for object detection. Additionally, YOLOv6 enhanced hardware utilization, making the model more effective for real-world deployment. The neck design in YOLOv6 features the newly introduced reparameterized path aggregation network (Rep-PAN), an innovation based on hardware-aware neural network principles. This design supports high-speed inference on various hardware platforms, addressing the latency issues introduced by the decoupled head structure of YOLOX. The decoupled head in YOLOv6 was redesigned with a hybrid channels strategy to balance representation capability and computational overhead. This modification reduces latency while maintaining high accuracy, mitigating the delays typically associated with 3×3 convolution operations in decoupled head architectures. This careful balance between computational efficiency and detection accuracy is a hallmark of YOLOv6.

To further optimize computational and memory efficiency, YOLOv6 introduced a CSP module within the backbone. This module divides the input feature map into two parts, applies independent convolutional operations to each, and then merges the output feature maps before passing them to the next backbone stage. This design reduces the number of parameters and computational requirements while preserving multi-scale feature extraction. By doing so, YOLOv6 achieved better performance without increasing resource demands, making it highly suitable for large-scale industrial applications. Another notable innovation in YOLOv6 is the adoption of the scaled IoU (SIoU) loss function for bounding box regression. SIoU redefines distance loss by incorporating vector angles between predicted and ground truth bounding boxes. This approach reduces the degrees of freedom in regression, accelerates network convergence, and improves the precision of bounding box localization. Experiments designed to test individual components of the model had shown that SIoU significantly enhances the model's mAP and FPS, confirming its effectiveness in improving regression accuracy. YOLOv6 also incorporated ensemble learning to boost accuracy. This technique involves training multiple models with different architectures and hyperparameters on the same dataset. The outputs of these models are then combined using weighted averages to produce the final predictions. Ensemble learning enables YOLOv6 to achieve higher accuracy than single-model approaches, making it particularly effective for challenging detection tasks. In summary, YOLOv6 introduced a series of innovations, including the EfficientRep backbone, Rep-PAN neck, CSP modules, SIoU loss, and ensemble learning. These features collectively enhance the model's accuracy, speed, and computational efficiency, solidifying YOLOv6 as a robust solution for industrial object detection applications.

8) YOLOv7

In July 2022, the official YOLOv4 team launched YOLOv7 [66], which achieved a significant milestone by attaining

an mAP of 56.8% on the COCO2017 dataset. YOLOv7 introduced the CSPVOVNet model, an advancement based on VOVNet [67], with notable improvements in parameter efficiency, computational complexity, memory access cost, and the input-output channel ratio. By optimizing these aspects, YOLOv7 enhanced the overall network's ability to perform faster and more accurate inference during training and testing. A novel method for analyzing gradient flow paths was also introduced, enabling the model to assign diverse feature weights to different layers, further improving feature representation. YOLOv7 innovatively addressed label assignment challenges in network training. Traditionally, models assign hard labels based on predefined rules, directly aligning outputs with ground truth values. However, YOLOv7 introduced a label classifier mechanism that generates "soft labels" by dynamically combining prediction outputs with ground truth values. This approach allows shallow auxiliary heads to learn from the guiding head's knowledge, effectively improving the learning process. By dynamically adjusting the importance of fine and coarse labels during training, YOLOv7 ensures a more reasonable generation of soft labels, improving the network's overall learning efficiency. To optimize parameter usage and computational efficiency, YOLOv7 incorporated model extension and composite scaling methods. These techniques balance scaling factors such as input size, depth, width, and stage to manage parameters and improve computational complexity, inference speed, and detection accuracy. This balance reduces the model's parameter count while enhancing both inference speed and detection precision, making YOLOv7 a robust solution for high-performance applications.

A key innovation in YOLOv7 is the introduction of the extended efficient layer aggregation network (E-ELAN) method. Built upon the original effective layer aggregation network (ELAN), E-ELAN improves the efficiency of deeper networks by controlling the shortest and longest gradient paths. By utilizing expansion, shuffling, and merging cardinality, E-ELAN enhances network learning capabilities without disrupting the gradient flow. This efficient long-range attention mechanism enables YOLOv7 to achieve faster convergence and better performance on deeper networks. In summary, YOLOv7's advancements, including CSPVOVNet, the label classifier mechanism, composite scaling methods, and E-ELAN, contribute to its superior balance of speed, efficiency, and accuracy. These innovations allow YOLOv7 to outperform previous models in both inference speed and detection performance, making it a critical milestone in the evolution of YOLO models.

9) YOLOv8

In 2023, the Ultralytics team, known for developing YOLOv5, released YOLOv8, a state-of-the-art detection algorithm designed for quick deployment and scalability. YOLOv8 retains the flexibility of offering multiple model variants to cater to different performance and resource requirements: YOLOv8n (nano), YOLOv8s (small),

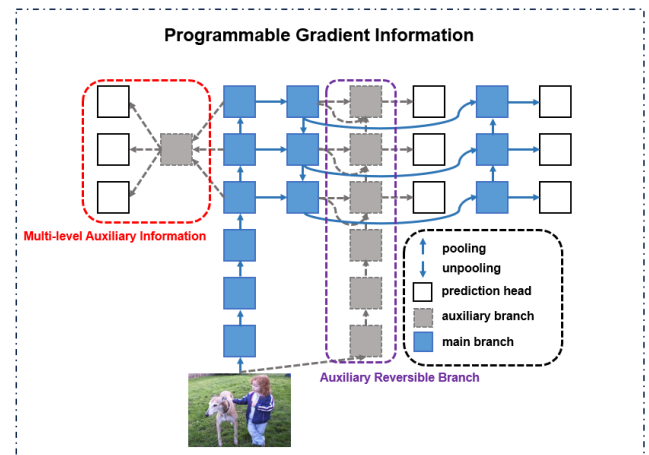


FIGURE 18. The PGI mechanism [40]. PGI consists of three parts: main branch, auxiliary reversible branch, and multi-level auxiliary information.

YOLOv8m (medium), and YOLOv8l (large). These models feature a lighter network structure and employ more efficient inference techniques, such as tensor runtime (TensorRT) engine acceleration, which is a high-performance deep learning inference library developed by NVIDIA, resulting in significantly improved inference speed compared to YOLOv7. YOLOv8 demonstrated an impressive increase in speed while maintaining high detection accuracy, making it an ideal choice for real-time applications. In addition to speed, YOLOv8 achieved a notable improvement in accuracy. On the COCO dataset, the mAP score of YOLOv8 is approximately 2% higher than that of YOLOv5. This improvement is attributed to more advanced training methodologies and optimization techniques, which shorten training time, accelerate convergence, and enhance the model's generalization capabilities. YOLOv8 also introduced a wider range of hyperparameters and customizable model structure options, enabling users to tailor the model to specific tasks and optimize performance with greater ease.

One of YOLOv8's most significant innovations is its support for diverse deployment platforms. In addition to CPUs and GPUs, YOLOv8 is optimized for edge devices such as Jetson Nano [68] and Raspberry Pi [69], as well as mobile devices like smartphones and tablets. This adaptability makes YOLOv8 suitable for a wide range of application scenarios, from high-performance industrial systems to resource-constrained environments. In summary, YOLOv8 builds upon its predecessors by combining enhanced inference speed, improved accuracy, and broader deployment versatility. With its lighter architecture, advanced training techniques, and support for edge and mobile devices, YOLOv8 is well-positioned to address the diverse needs of modern object detection tasks.

10) YOLOv9

In February 2024, Wang et al. [40] introduced YOLOv9, establishing a new state-of-the-art detection model. Building on the contributions of YOLOv4 and YOLOv7, YOLOv9

tackled a critical challenge in deep learning models: the information bottleneck. During feature extraction and spatial transformation, significant information loss can occur, particularly in tasks involving lightweight models or complex scenarios. Existing methods to address this issue, such as reversible architectures [70], mask modeling [71], and deep supervision [72], have limitations. These approaches often result in higher inference costs or fail to effectively address information loss in certain contexts. To overcome these drawbacks, YOLOv9 introduced a novel mechanism called programmable gradient information (PGI), as shown in Fig. 18. The PGI mechanism adds two key components to the main branch of the network: an auxiliary reversible branch and multi-level auxiliary information. The auxiliary reversible branch uses an auxiliary supervision mechanism to generate useful gradients for updating network parameters. This approach allows the network to avoid forcing the main branch to retain complete original information, making it effective even in shallow networks. Importantly, the auxiliary reversible branch is removed during inference, ensuring that it does not increase inference costs. Multi-level auxiliary information, on the other hand, aggregates gradient information across all objects and passes it to the main branch. This prevents the feature pyramid hierarchy from being dominated by specific object information, reducing the risk of information leakage. Furthermore, the multi-level auxiliary information mechanism is compatible with deep neural networks of various scales, providing a universal solution to challenges such as information loss and increased inference times.

YOLOv9 also introduced a generalized effective layer aggregation network (GELAN), built upon the ELAN framework, to optimize parameters, inference speed, and accuracy. By combining elements of CSPNet and ELAN, GELAN enhances feature aggregation and model efficiency. This design reduces computational overhead while maintaining high detection precision. On the COCO dataset, YOLOv9 achieved superior accuracy compared to its predecessors while significantly reducing the number of parameters and computational complexity. In lightweight model comparisons, YOLOv9 reduced the number of parameters by 10% and improved the mAP by approximately 0.5% compared to the prior best-performing YOLO model. For large-scale models, YOLOv9-E demonstrated remarkable improvements over YOLOv8-X, reducing parameters by 15%, lowering computational complexity by 25%, and achieving a 1.7% increase in mAP. These advancements highlight YOLOv9's effectiveness in addressing both performance and efficiency challenges. In summary, YOLOv9's innovative PGI mechanism and GELAN architecture enable it to overcome information bottlenecks while enhancing accuracy and speed. By combining flexibility, scalability, and efficiency, YOLOv9 sets a new benchmark for object detection models and reinforces its position as a cutting-edge tool for diverse applications.

11) YOLOv10

YOLOv10, developed by Wang et al. [41] from Tsinghua University in May 2024, introduced several advancements that enhance both accuracy and efficiency, making it particularly well-suited for real-time applications. One of the key innovations in YOLOv10 is its dual label allocation strategy, which optimizes the matching process between “one-to-many” and “one-to-one” label assignments. This dual approach balances training accuracy and speed, resulting in more effective learning. Additionally, it eliminates the need for NMS during the inference process, simplifying real-time deployment and significantly reducing post-processing latency. To further optimize computational efficiency, YOLOv10 incorporated lightweight classification heads and employed separable convolutions. By splitting spatial and channel adjustments during downsampling, YOLOv10 preserved essential image details required for fast detection while minimizing computational costs. The model selectively applies large kernel convolutions in deeper layers to enhance performance without introducing delays. Moreover, the inclusion of the partial self-attention (PSA) mechanism improves global feature extraction with minimal computational overhead, ensuring the network can capture broader context efficiently. These architectural enhancements translate into tangible performance improvements. Compared to its predecessors, YOLOv10 achieved an increase of 1.2% to 1.4% in mAP while reducing latency by up to 70% across various model scales. The streamlined architecture of YOLOv10 further reduced parameters by 68% compared to models like YOLOv9, making it exceptionally lightweight and efficient. This reduction in complexity, combined with its high accuracy, makes YOLOv10 an ideal choice for resource-constrained environments. The design of YOLOv10 is particularly suited for real-time applications with hardware limitations, including mobile computing platforms and autonomous systems. By maintaining a balance between computational efficiency and detection performance, YOLOv10 sets a new standard for object detection in scenarios where both speed and precision are critical.

III. APPLICATIONS OF YOLO ALGORITHMS

A. YOLO IN THE TRAFFIC SECTOR

YOLO algorithms have demonstrated exceptional potential in the traffic sector, particularly in areas such as pedestrian detection, autonomous driving, and traffic sign recognition.

1) YOLO FOR PEDESTRIAN DETECTION

In 2018, Lan et al. [73] introduced the YOLO-R model, a novel enhancement of the YOLOv2 architecture designed to improve pedestrian detection. The model integrated three additional pass-through layers beyond the original sixteen layers and extended the connectivity of the routing layers of YOLOv2. These enhancements allow for better feature utilization across layers, resulting in significant performance improvements. Validation of the model on the INDIA

pedestrian detection dataset revealed an increase in detection accuracy from 97.37% to 98.56%, alongside a reduction in the missed detection rate to 10.05%.

2) ADVANCING AUTONOMOUS DRIVING WITH YOLO

YOLO models are also widely applied in autonomous driving, particularly in 3D object detection systems. In 2021, researchers developed a real-time 3D object detection system tailored for autonomous driving scenarios, leveraging light detection and ranging (LiDAR) and state-of-the-art YOLOv4 for 2D object detection in RGB images [74]. This system generated multi-class 3D bounding boxes using complex regression techniques, enabling precise pose predictions. It processes point cloud data to produce real-time Euler region proposal networks (E-RPN) and class-based 3D bounding boxes. Experimental results on the Karlsruhe Institute of Technology and Toyota Technological Institute (KITTI) benchmark dataset [75] showed that the system outperformed existing methods in both accuracy and performance, demonstrating its viability for real-time applications in autonomous vehicles.

3) ENHANCING TRAFFIC SIGN RECOGNITION USING YOLO

In 2022, YOLO algorithms were further utilized for traffic sign recognition. A convolutional neural network based on YOLOv4-Tiny, a lightweight version of YOLOv4, was developed to enhance traffic sign recognition [76]. The performance of the system was rigorously evaluated, achieving an impressive recognition accuracy of 99.4%. This highlights the effectiveness of YOLO in handling small objects and achieving precise detections in traffic scenarios.

B. YOLO IN AGRICULTURE

The YOLO object detection framework has demonstrated significant potential in addressing various challenges in agriculture. From crop picking to pest detection, YOLO has become a valuable tool in improving agricultural efficiency and precision.

1) CROP PICKING

In crop harvesting, YOLO-based algorithms had been employed to address challenges such as occlusion, uneven lighting, and shadow distribution. In 2021, Lawal [77] proposed the YOLO-Tomato model, an improvement on the YOLOv3 network. This model incorporated several enhancements, including a label visibility method, dense architecture merging, SPP, and the Mish activation function. YOLO-Tomato was further divided into three variations as models A, B, and C, based on their activation functions. Model A used the Leaky ReLU activation, while models B and C employed Mish and Mish with SPP, respectively. Testing on a custom tomato dataset demonstrated that YOLO-Tomato-C achieved an impressive accuracy of 99.5%, outperforming other detection algorithms in complex environmental conditions.

2) PEST DETECTION

YOLO algorithms have also been successfully applied to pest identification. In 2022, Pang et al. [78] developed the

YOLOv4-PEST model, an enhanced version of YOLOv4, to identify orchard pests. Improvements included upgrading the NMS method to DIoU NMS and introducing data augmentation with two rounds of fine-tuning. The model achieved an mAP of 92.86% and a detection speed of 12.22 ms on the custom PestImgData dataset, showcasing its efficiency in identifying pests in orchards. This innovation offers a practical solution for pest monitoring and management, contributing to increased crop yields and reduced losses.

3) PESTICIDE SPRAYING WITH DRONES

To address the challenges of pesticide spraying in rugged environments such as small fields or mountainous regions, Qin et al. [79] developed the Ag YOLO algorithm, an improved version of YOLO. Ag YOLO was integrated with the Intel neural computing stick 2 (NCS2) [80], forming a lightweight onboard system for drones used in betel nut plantations in tropical regions. By modifying the YOLOv3-Tiny architecture, its neck and backbone were replaced with advanced models such as SqueezeNet [81], MobileNetv2 [82], and ShuffleNetv2 [83]. Ag YOLO achieved an accuracy of 76.55% and a speed of 36.5 FPS on the resource-constrained NCS2 hardware. This model reduced parameters by nearly 12 times compared to the original YOLOv3 Tiny, while doubling its speed, demonstrating its suitability for resource-limited applications.

4) BROADER AGRICULTURAL APPLICATIONS

Beyond these specific use cases, YOLO offers wide-ranging benefits in agriculture. For instance, YOLO can automate fruit and vegetable harvesting by detecting their locations and guiding harvesters to the correct positions, thereby reducing waste and improving efficiency. Additionally, YOLO can assist farmers in monitoring and managing crops by detecting and diagnosing plant diseases and pests. This enables timely interventions to prevent crop damage, ultimately increasing yields. The automation of processes such as crop detection and harvesting also saves farmers time and reduces operational costs, making agriculture more sustainable and profitable.

C. YOLO IN THE MEDICAL AND HEALTHCARE SECTOR

YOLO-based object detection algorithms have demonstrated their potential in advancing medical imaging, disease diagnosis, and healthcare monitoring, addressing some of the sector's most critical challenges as follows.

1) PULMONARY NODULE DETECTION AND EARLY LUNG CANCER DIAGNOSIS

In 2018, Ramachandran et al. [84] enhanced the traditional DetectNet framework by integrating YOLOv1, a CNN-based object detection model. This approach replaced manual feature recognition networks, enabling the early detection and recognition of pulmonary nodules by analyzing their

shape, size, and characteristics. The study utilized the lung image database consortium (LIDC) dataset [85], training the model with NVIDIA DIGITS and the Caffe deep learning frameworks. The enhanced YOLOv1-based method demonstrated a reduced false alarm rate, highlighting its efficacy in medical image analysis for early-stage disease detection and improving lung cancer survival rates.

2) ADVANCEMENTS IN CT IMAGE ANALYSIS WITH GANs AND YOLO

In 2020, Hammami et al. [86] proposed a novel combination of cyclic generative adversarial network (CycleGAN) [87], a GAN for unsupervised learning, with the YOLO object detection algorithm. This cross-modal synthesis technique utilized CycleGAN to generate synthetic computed tomography (CT) images from magnetic resonance imaging (MRI) data, which were then annotated and used to enhance YOLO's detection capabilities. Using the visceral anatomy benchmarks dataset [88], this method significantly improved the detection and recognition of multiple organs in 3D CT images. With an average detection distance of 7.95 ± 6.2 mm, this approach outperformed YOLO alone and proved superior to other state-of-the-art medical image detection methods.

3) HEART DISEASE PREDICTION USING LBO-YOLOv4

In 2022, Alamelu et al. [89] introduced the lion-based butterfly optimization (LBO)-YOLOv4 algorithm, an enhancement of YOLOv4, for predicting sensor data collected via the internet of medical things (IoMT). The study also utilized an improved CSPDarknet-53-based YOLOv4 algorithm to classify echocardiography images, effectively predicting heart disease. The model achieved impressive accuracy rates, with 97.25% for regular sensor data and 98.87% for irregular sensor data. This dual-method approach demonstrated its effectiveness in heart disease detection, offering a reliable tool for medical diagnostics.

4) COVID-19 SOCIAL DISTANCING MONITORING

During the COVID-19 pandemic, maintaining social distancing became a critical measure for controlling virus transmission. In this context, Punnett et al. [90] fine-tuned the YOLOv3 network with deep simple online and realtime tracking (DeepSORT) technology to detect and track individuals in real time. By comparing various deep learning object detection frameworks, the study identified YOLO as the most effective detection network for this task. Using bounding boxes and assigned identifiers (IDs), YOLO and DeepSORT enabled real-time monitoring of social distancing. This system effectively balanced mAP and FPS scores, offering a robust solution for mitigating COVID-19 spread.

YOLO's ability to handle diverse medical applications, from early disease diagnosis to pandemic management, underscores its value in the healthcare sector. By leveraging innovative enhancements like GAN integration, advanced network architectures, and real-time monitoring capabilities,

YOLO continues to provide critical solutions for improving medical and healthcare outcomes.

D. YOLO IN INDUSTRIAL SETTINGS

YOLO-based object detection algorithms have found significant applications in industrial environments, improving the efficiency and accuracy of processes such as electronic component detection, solar cell defect identification, and retail packaging testing.

1) ELECTRONIC COMPONENT DETECTION

In 2021, Guo et al. [91] proposed a novel detection model based on an improved YOLOv4-Tiny architecture to enable intelligent robots to detect electronic components moving at high speeds on conveyor belts. To enhance the model's ability to recognize mid- and high-level features, the authors integrated two attention modules: the dual channel attention (DCA) module [92] and the batch normalized channel attention (BNCA) module, inspired by multi-style attention fusion network (MAFNet) [93]. This fusion improved the detection performance across different feature levels. Additionally, the use of a multi-scale attention module (MAM) filtered out erroneous features, further enhancing detection accuracy. The validation on a custom electronic component dataset demonstrated a detection accuracy of 98.6%, significantly outperforming other mainstream models like SSD and Fast-RCNN. This high accuracy underscores the effectiveness of YOLO-based models in fast-paced industrial processes.

2) SOLAR CELL DEFECT DETECTION

In 2021, Zhang et al. [94] introduced an improved YOLOv5 algorithm for identifying defects in solar cells. To enhance feature extraction, the authors incorporated the ECA-Net attention mechanism [95], addressing issues of information redundancy caused by background clutter. A variable convolution module was also added to improve the model's adaptive learning capabilities. This enhanced YOLOv5 algorithm achieved an mAP of 89.64% on a solar cell electroluminescence (EL) image dataset with an improvement of 7.85% over the original algorithm. With a processing speed of 36.24 FPS, the model effectively handled challenges such as complex backgrounds, varied defect morphologies, and scale differences in solar cell images. This advancement enables real-time and accurate detection of solar cell defects, addressing critical quality assurance needs in the solar industry.

3) RETAIL PACKAGING TESTING

In 2023, Pham et al. [96] applied YOLOv5 to detect defects in retail packaging during factory production. Using a dataset consisting of 200 images of damaged and intact packaging boxes, the model was trained over 150 rounds. The resulting system achieved an mAP of 78.5% for packaging box detection, demonstrating its potential to improve quality control in retail packaging lines. This application highlights

YOLOv5's adaptability to diverse industrial tasks, even in environments with limited datasets.

YOLO algorithms have proven invaluable in industrial applications by offering high accuracy and efficiency. From electronic component detection to solar cell quality assurance and retail packaging inspections, YOLO models address critical challenges in fast-paced and complex industrial environments. These advancements not only improve production quality but also meet real-time operational demands, reinforcing YOLO's role as a cornerstone in industrial automation.

E. YOLO IN REMOTE SENSING

YOLO-based object detection algorithms have been extensively applied in remote sensing to address challenges in detecting resources, managing natural disasters, and analyzing urban infrastructure. These models demonstrated exceptional performance in processing high-resolution satellite images and aerial views.

1) DETECTION OF OIL WELL RESOURCES

In 2021, Shi et al. [97] proposed an improved object detection algorithm based on YOLOv4 for identifying oil well resources in satellite imagery. The study addressed issues such as repeated detections and inaccurate positioning by employing a sliding slice method and discarding edge detections. This approach utilized a sliding window technique to process large-scale satellite images while eliminating detection results within a specified edge width, improving overall accuracy. Given the lack of high-resolution datasets, an oil well dataset was created using Google Earth images with a spatial resolution of 0.26 meters. The experimental results demonstrated that the improved YOLOv4 method achieved a detection accuracy of 94.7%, outperforming SSD's accuracy of 93.2%, particularly excelling in small object detection tasks.

2) FLOOD DISASTER MANAGEMENT AND SURVIVOR RESCUE

In 2022, Prabhu et al. [98] introduced RescueNet, a model based on the YOLO architecture, designed to aid in rescuing survivors during floods. The dataset used for training included aerial views of human and animal targets, supplemented by images from the Unmanned Aerial Vehicle Laboratory at the Indian Institute of Science, Bangalore, India. RescueNet employed a 21-layer network structure for extracting feature maps and utilized a 32x downsampling technique to detect smaller targets in densely populated areas. The model achieved an mAP of 98.09%, surpassing the performance of YOLOv3 and YOLOv4 while doubling detection speed. This innovation highlights the potential of YOLO-based models in real-time disaster response and rescue operations.

3) URBAN GAS STATION RECOGNITION

In another application of YOLO in remote sensing, Gao et al. [99] developed the YOLO-S-CIOU (YOLO, S, and CIOU

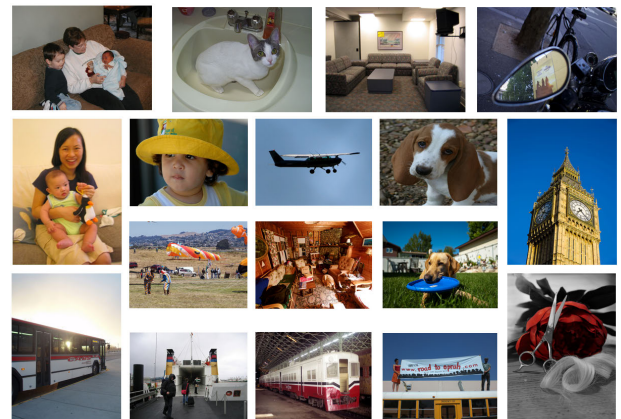


FIGURE 19. Example images for object detection dataset. These images are selected from the VOC2007 dataset [29].

stand for YOLOv3, SRXnet, and CIOU loss, respectively) model for recognizing gas stations in urban areas. This model, an improvement on YOLOv3, replaced the Darknet-53 backbone with SRXnet (constructed by superimposing multiple SE-ResNeXt), a network incorporating multiple squeeze-and-excitation residual networks with next generation (SE-ResNeXt) [100] modules to enhance feature learning. The authors also employed CIOU as the loss function to improve bounding box regression accuracy. Testing on a custom urban gas station dataset revealed that YOLO-S-CIOU improved the mAP by 2.33% compared to YOLOv3, while simultaneously reducing model parameters. These enhancements resulted in stronger robustness and improved detection capability across diverse regions in remote sensing tasks.

YOLO-based algorithms in remote sensing have proven instrumental in tackling a wide range of challenges, from detecting natural resources like oil wells to managing disasters and analyzing urban infrastructure. With innovative improvements in architecture and training techniques, these models provide enhanced accuracy and efficiency, making them indispensable tools for geospatial analysis and real-world applications.

IV. METHODOLOGY

A. DATASETS

The remarkable progress of deep learning algorithms is closely tied to the availability of large-scale datasets. Object detection algorithms, in particular, rely on these datasets for performance evaluation, algorithm comparison, and fostering innovation in addressing real-world complexities. Object detection datasets typically consist of images or videos annotated with labeled objects, bounding boxes, segmentation masks, or key points, serving as benchmarks for evaluating algorithm performance. Figure 19 illustrates several example images from commonly used object detection datasets.

Table 3 lists the main information about representative datasets and their open-source websites. The PASCAL VOC Challenge Competition, first held in 2005, marked

TABLE 3. List of specialized datasets for computer vision: sources, web links, and numbers of image quantities provided for diverse applications.

Dataset	URL	Number
VOC2007	http://host.robots.ox.ac.uk/pascal/VOC/voc2007/	~10,000
COCO2017	https://cocodataset.org/	~328,000
ImageNet	https://storage.googleapis.com/openimages/web/index.html	14,197,122
DOTAv1	https://captain-whu.github.io/DOTA/dataset.html	188,282
Flame and smoke	https://www.kaggle.com/datasets/dataclusterlabs/fire-and-smoke-dataset	~7,000
Waste	https://www.kaggle.com/datasets/dataclusterlabs/domestic-trash-garbage-dataset	~9,000
Pedestrian	https://mmlab.ie.cuhk.edu.hk/datasets/cuhk_occlusion	1,063
Signal Light	https://cg.cs.tsinghua.edu.cn/traffic-sign/	100,000
Text Detection	https://vision.cornell.edu/se3/coco-text-2/	63,686

a milestone in computer vision by introducing one of the earliest object detection datasets. Although the competition was suspended in 2012 and the dataset ceased updating, the combination of VOC2007 and VOC2012 datasets remains a widely used benchmark dataset for evaluating object detection algorithms. Similarly, ImageNet, created in 2007 by Li et al., revolutionized visual object recognition research. ImageNet containing 14,197,122 images across 1,000 categories was collected via web crawling, manual labeling, and Amazon crowdsourcing. Introduced at the 2009 computer vision and pattern recognition (CVPR) conference, the ImageNet inspired the ImageNet large scale visual recognition challenge (ILSVRC) in 2010, where breakthrough CNN models like AlexNet [101], OverFeat [102], GoogleNet, VGG, ResNet, and SENet [100] showcased their potential. In 2014, Microsoft released the Microsoft COCO dataset, a benchmark dataset for object detection. The COCO challenge, which ended in 2017, used the widely adopted COCO2017 dataset. Another influential dataset, Open Images, was launched by Google in 2016. It features approximately 9 million images divided into 6,000 classes, with more real-life entities than ImageNet's 1,000 classes. The latest version, OpenImagesV7, released in October 2022, contains 66.4 million point-level labels spanning 1.4 million images and 5,827 categories. The OpenImages dataset provides sparse pixel-level localization and is continually updated to meet evolving research needs.

Apart from general-purpose datasets, certain datasets are tailored for specific domains. For instance, the DOTA dataset (introduced in 2018) is pivotal for remote sensing object detection [103]. Designed for aerial imagery, DOTA includes large-scale photographs with diverse scales, orientations, and shapes. The original dataset had 15 object classes, including vehicles and basketball courts, with image sizes ranging from 800×800 to $20,000 \times 20,000$ pixels. The latest version was expanded to 18 categories, reflecting advancements in aerial imaging technology. Another specialized dataset, the flame and smoke image dataset [104], is vital for early fire and smoke detection. This dataset includes

7,000 images captured during real fire scenarios, such as garbage incineration, paper and plastic burning, crop field incineration, and home cooking. Similarly, the domestic waste dataset [105], comprising 9,000 images, focuses on commonly seen waste types in India under various lighting and environmental conditions, supporting applications like garbage detection, eco-friendly alternatives, and carbon footprint estimation. Additional datasets cater to specific object detection tasks, including crowd datasets, scene datasets, and text detection datasets. These datasets expand the scope of object detection applications across diverse domains.

For our benchmark testing, we selected both the PASCAL VOC 2007 and 2012 (VOC07+12) dataset and the COCO2017 dataset, representing small and large datasets, respectively. Both datasets are widely recognized in object detection projects for their authoritative annotations and open-source availability. The VOC07+12 dataset offers annotations for a limited set of categories, while the MS COCO2017 dataset provides an extensive variety of labeled objects, making these datasets ideal for training, validating, and testing object detection algorithms in practical application scenarios.

1) THE PASCAL VOC07+12

One well-known image recognition dataset specifically created for object detection and classification tasks is PASCAL VOC07+12. It was first created for the 2005 PASCAL VOC Challenge Competition, which evaluated sophisticated methods for locating and identifying things in photos. Numerous excellent object recognition and segmentation models were produced as a result of this competition. Researchers can still submit prediction results to the server to assess the effectiveness of their models, even if the PASCAL VOC Challenge Competition ended in 2012. Recent detection or segmentation publications frequently employ the VOC 2007 and 2012 datasets (VOC07+12), which are crucial for researchers. There are 16,551 photos with 20 classes in the training set and 4,952 images in the test set.

TABLE 4. Comparative specifications of NVIDIA GPU models showcasing configuration information for each GPU.

GPUs	TFLOPS (Single-Precision)	Memory Size (GB)	Memory Type	Bandwidth (GB/s)	CUDA Cores
GTX Titan X	6.7	12	GDDR5	336.6	3,072
RTX 3060	12.7	12	GDDR6	360	3,584
Tesla V100	15.7	32	HBM2	900	5,120

2) THE MICROSOFT COCO2017

Like the ImageNet competition, the Microsoft COCO2017 is considered as one of the most important and prestigious computer vision contests. It was first sponsored by Microsoft in 2014. It was a sizable and varied dataset suitable for segmentation, object recognition, and captioning. Of the 123,287 images on the COCO2017 dataset, 5,000 are in the validation set and 118,287 are in the training set. Of the 80 object types in the sample, 41.37% are small objects, 33.86% are medium objects, and 24.77% are large objects. Although there are fewer categories than in the ImageNet dataset, each category contains more photos, making it easier to find certain objects within each category. The COCO2017 dataset contains more categories and images than the PASCAL VOC07+12 dataset, and this makes it appropriate for testing more sophisticated and complicated models that can manage numerous object instances and segmentation tasks. Therefore, evaluating the accuracy of the model after being trained on the COCO2017 dataset has emerged as a critical criterion for judging the quality of an algorithm. In the study, to ensure a thorough evaluation of the performance, all models have been trained on the training set and assessed on the validation set of the COCO2017 dataset, following the official YOLO versions.

B. HARDWARE PLATFORMS

Object detection models have seen remarkable advancements, partly due to the increased integration of GPUs in their implementations. Initially designed for gaming and visualization applications, GPUs have become indispensable for accelerating deep learning tasks due to their parallel processing capabilities. Their architecture enables efficient computation of large-scale neural networks, significantly enhancing the performance of object detection algorithms and facilitating real-time applications across various domains. While standard GPUs are sufficient for general detection tasks involving small datasets or a limited number of object classes, they often struggle with the computational demands of multi-class object detection and large dataset training in competitive tasks.

1) CLOUD COMPUTING AND GPU ACCESSIBILITY

Cloud computing platforms provide access to state-of-the-art GPU hardware from manufacturers such as NVIDIA, effectively addressing the limitations of standard GPUs. By renting GPUs, researchers can leverage advanced hardware features, including high-speed memory, sophisticated parallel

processing units, and hardware-accelerated deep learning frameworks, enabling faster training times. Purchasing high-performance GPUs, however, can be cost-prohibitive. For instance, the GTX Titan X was priced at approximately USD 999 when released in 2015, while the Tesla V100 32 GB was priced at about USD 10,000 when first released in 2018. Renting GPUs on cloud platforms offers a cost-effective alternative, with fees varying based on the model and usage duration. For example, the price in 2024 of renting a Tesla V100 on the Google Cloud Platform ranges from USD 1.46/hour to USD 3.55/hour. Properly selecting a high-end GPU and managing the rental duration based on training time can minimize costs, making cloud computing an attractive option for researchers with budget constraints.

2) GPU SPECIFICATIONS AND CAPABILITIES

The GTX Titan X [106], introduced by NVIDIA in March 2015, is a high-performance graphics card designed for computationally demanding tasks. Its ability to process significant amounts of data in parallel across thousands of cores makes it suitable for deep learning applications, enabling faster training times for large and complex datasets. The GTX Titan X features 12 GB of graphics double data rate (GDDR) type GDDR5 memory and 3,072 compute unified device architecture (CUDA) cores, optimized for high-performance computing and gaming. With a memory bandwidth of 336.5 GB/s and a clock speed ranging from 1,000 MHz (base) to 1,075 MHz (boost), it can deliver robust single-precision (FP32) performance, peaking at 6.7 Tera floating point operations per second (FLOPS) or TFLOPS. While its double-precision (FP64) performance was minimal (1/32 of FP32), it excels in tasks such as deep learning, 3D rendering, and complex simulations. The GTX Titan X remains a popular choice for researchers due to its impressive performance in accelerating neural network training. The ray racing texel extreme RTX 3060, part of NVIDIA's 30-series lineup, released in February 2021, is based on the Ampere architecture. With 3,584 CUDA cores, 12 GB of GDDR6 memory, and the memory bandwidth of up to 336.6 GB/s, it delivers powerful computational performance. Its performance of 12.7 TFLOPS makes it well-suited for training neural networks and processing large datasets. The RTX 3060's combination of speed, memory capacity, and cost-efficiency makes it an attractive choice for researchers and teams seeking reliable performance without incurring high costs. The Tesla V100 [108], introduced by NVIDIA in March 2018, is a high-performance GPU specifically

TABLE 5. Detailed specifications of the platforms hosting the NVIDIA GTX Titan X, RTX 3060, and Tesla V100 GPUs.

Hosts	GPUs	CPU Models	CPU Cores	RAM (GB)	Interface
Personal computer	GTX Titan X	Intel Core i7 10700	4 (8)	32	PyCharm
Personal computer	RTX 3060	Intel Core i7 10700	4 (8)	32	PyCharm
NCHC cloud computer	Tesla V100	Intel Xeon Platinum 8280	4 (28)	96	Terminal

designed for compute-intensive applications such as deep learning. Based on the Volta architecture, the Tesla V100 features 5,120 CUDA cores, 640 Tensor cores, and 32 GB of the second generation high-bandwidth memory (HBM2) with a memory bandwidth of up to 900 GB/s. Its peak single-precision floating-point performance of 15.7 TFLOPS and support for mixed-precision arithmetics make it a powerhouse for training deep neural networks and processing extensive datasets. The Tesla V100 is a staple in deep learning research and is frequently utilized in YOLO-related studies, underscoring its relevance in object detection. The detailed parameters of the three GPUs are listed in Table 4. GPUs such as the Tesla V100 are optimized for deep learning, with tensor cores, higher memory bandwidth, and faster computing power. The tensor cores and memory bandwidth of GPUs greatly affect the training speed of deep models such as YOLO. Compared with older models such as the GTX Titan X, the YOLO training speed on the Tesla V100 GPU is faster.

3) BENCHMARKING GPU CONFIGURATIONS FOR YOLO

Training YOLO models, whether on purchased GPUs or rented cloud platforms, involves considerable expenses. Thus, selecting an appropriate GPU hardware platform and estimating the training time required for YOLO models are crucial to saving costs. To provide a comprehensive understanding of YOLO training benchmarks across different hardware configurations, we selected three GPUs with detailed specifications for this study. We evaluate the performance of various YOLO versions on these three different GPU configurations. These include the GTX Titan X, a popular high-end GPU from 2015, the affordable RTX 3060 commonly used for gaming, and the high-performance Tesla V100, designed specifically for deep learning and scientific computing applications.

The hosts for the GTX Titan X and the RTX 3060 GPUs are both personal computers, and for the Tesla V100 we chose to rent a cloud computer with the Tesla V100 GPU provided by the National Center for High-performance Computing (NCHC) in Taiwan. The CPU models of the hosts corresponding to the first two GPUs are both Intel Core i7 10700, while that for the host with the Tesla V100 is Intel Xeon Platinum 8280. The number of CPU cores is 8 on the hosts of the GTX Titan X and RTX 3060, while the platform hosting the Tesla V100 has 28 cores although we use 4 out of their total number of cores in the benchmark test. In terms of computer memory, the memory sizes of the computers with GTX Titan X and RTX 3060 are both 32 GB. The memory size on the Tesla V100 platform is three times that

of the other two, which is 96 GB. In terms of the interface used, the Tesla V100 is trained directly on the terminal, while the GTX Titan X and the RTX 3060 are both trained using PyCharm software. This evaluation aims to generalize YOLO's performance across a wide range of hardware configurations, catering to researchers and practitioners with varying budgets and hardware resources. By analyzing training performance and speed on conventional datasets, this study offers valuable insights into optimizing YOLO training across diverse environments. The detailed specifications of the platforms hosting these three GPUs are listed in Table 5.

C. EXPERIMENTAL SETTINGS

The selection of YOLO versions for the benchmark test involved meticulous consideration of factors such as usage frequency, ease of deployment, and performance relevance. To ensure consistency and avoid fragmented comparisons, only official YOLO versions were included. YOLO-inspired models, such as PaddlePaddle-YOLO (PP-YOLO), You Only Learn One Representation (YOLOR), YOLO-neural architecture search (YOLO-NAS), Scaled-YOLOv4 were excluded due to lack of standardized training pipelines. YOLOv1 and YOLOv2 were excluded from testing due to their comparatively lower performance, accuracy, and limited usage. The benchmark tests were conducted using an Ubuntu 20.04 LTS environment with torch 1.8 and CUDA 11. The tested models included PyTorch YOLOv3 to YOLOv10, and YOLOX, which are widely employed in both academic and industrial applications. The environment configuration for YOLOv10 is torch 2.1.0. To ensure fairness and efficiency, the pre-trained weights used for the VOC07+12 and COCO2017 dataset benchmarks were the official pre-trained ones provided for each YOLO version. These pre-trained models, designed for the COCO2017 dataset, expedited convergence for other datasets, making them ideal for training customized datasets within limited epochs. However, since the YOLOv9 team has not open-sourced the lightweight YOLOv9-S model, we exclusively selected the largest YOLOv9-E model for benchmark testing. For the PASCAL VOC07+12 dataset, batch sizes were configured based on the memory capacities of the three selected GPUs. Specifically, batch sizes were set to 8 for YOLOv5-X, YOLOv7-X, YOLOv8-X, YOLOv10-X, and 4 for YOLOv9-E, all representing large YOLO models. For lightweight models, the batch size was uniformly set to 16. On the COCO2017 dataset, batch sizes were configured as 8 for YOLOv5-X, all YOLOv7 models, all YOLOv8 models,

TABLE 6. Evaluation of YOLO versions: comparison of model complexity in millions of parameters, computational cost in GFLOPs, and accuracy measured by mAP in percentage based on the training using the VOC07+12 dataset and the COCO2017 dataset. Here, the mAP values obtained by training each version of YOLO using the COCO2017 dataset are extracted from the official releases.

Version	Params (M)	GFLOPs	mAP _{50:95} (%) on VOC07+12	mAP _{50:95} (%) on COCO2017
YOLOv3	61.6	154.9	66.4	36.2
YOLOv4	64.0	142.2	53.6	43.5
YOLOv5-S	7.0	15.9	59.6	37.4
YOLOv5-X	86.3	204.2	74.1	50.7
YOLOX	9.0	26.8	63.5	40.5
YOLOv6	17.2	44.19	70.9	45.0
YOLOv7-S	37.3	105.4	71.2	51.4
YOLOv7-X	70.9	188.4	72.5	53.1
YOLOv8-S	11.1	28.7	67.6	44.9
YOLOv8-X	68.2	258.2	73.4	53.9
YOLOv9-E	70.0	245.5	76.0	55.6
YOLOv10-S	7.2	21.6	69.7	46.3
YOLOv10-X	29.5	160.4	74.0	54.4

YOLOv10-X, and 4 for YOLOv9-E. For other versions, we set it as 16. To ensure smooth training and mitigate errors due to CPU discrepancies, the number of workers were fixed at 4, enabling consistent completion of all training epochs across the three hardware platforms.

In terms of training epochs, we first trained all models on three GPU platforms for 100 epochs with both the VOC07+12 and COCO2017 datasets. The settings can facilitate a fair comparison of the time spent on each epoch of training across different platforms. Some models could reach convergent results in the first 100 epochs, and most of them were trained based on the VOC07+12 dataset. For other models that have not reached convergence, we later set more epochs for deeper training based on the convergence of the first 100 epochs to ensure that all models can reach convergence. Other parameter settings, such as learning rates, loss function methods, and related numerical configurations, were kept consistent with the original values outlined in the respective official YOLO projects which were released on the open sources to ensure comparability and reliability. This study primarily focuses on comparing training times across different GPU configurations, providing recommendations to optimize resource usage and reduce computational costs for YOLO users. These insights offer a practical guidance for deploying YOLO models effectively while balancing accuracy and speed.

V. THE BENCHMARK RESULTS AND DISCUSSION

A. OBJECT DETECTION PERFORMANCE ANALYSIS: mAP EVALUATION AND MODEL COMPLEXITY

An mAP is a critical metric for evaluating object detection models, measuring the AP across multiple classes at different

IoU thresholds, ranging from 0.5 to 0.95 ($mAP_{50:95}$). This ensures that the detection accuracy reflects the model's robustness across varying levels of overlap between predicted and ground truth bounding boxes. An IoU of 1 indicates perfect overlap, while an IoU of 0 represents no overlap. The inclusion of stricter thresholds, such as 0.95, tests the model's ability to predict highly accurate bounding boxes. The mAP metric, therefore, provides a comprehensive measure of detection accuracy. In addition to mAP, parameters (Params) and floating-point operations (FLOPs) are essential for evaluating the model's complexity and computational efficiency. Params indicate the size of the model, while FLOPs measure the computational cost required for inference. A larger parameter count often correlates with better accuracy but requires more computational resources, making the choice of a YOLO model dependent on the user's hardware capabilities and application requirements. Table 6 summarizes the mAPs for YOLO models trained on the VOC07+12 dataset and those on the COCO2017 dataset as released by their official versions along with the Params, FLOPs for each model. The YOLO versions developed by the same team are highlighted with the same color in Table 6 for easier identification.

1) PROGRESSION IN DETECTION ACCURACY

The benchmark results highlight a clear trend of improvement in detection accuracy as the YOLO architecture evolves. Early models like YOLOv3 achieved an mAP of 66.4% on VOC07+12 and 36.2% on COCO2017, with 61.6M Params and 154.9 GFLOPs. Although it set a baseline for object detection models, its performance is now significantly out-classed by newer versions. YOLOv4 brought improvements

AP of each Class on VOC07+12 Dataset

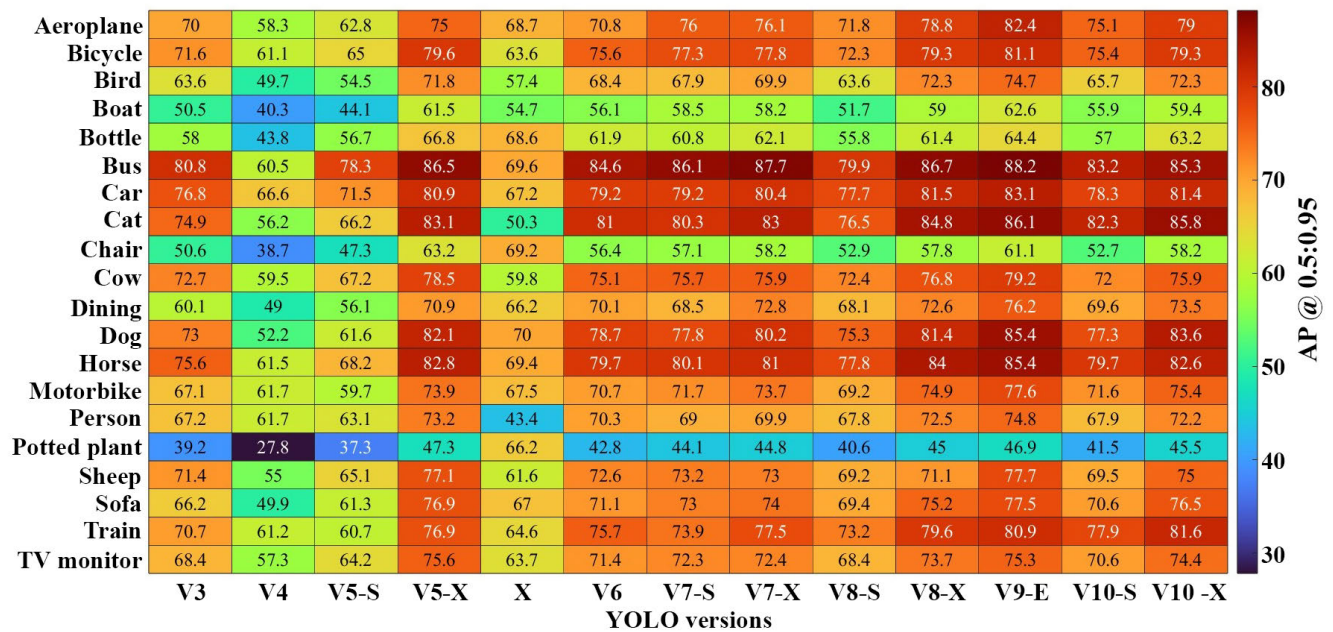


FIGURE 20. The AP values of 20 categories in the VOC07+12 dataset. In all 20 categories, the YOLOv9-E model achieved the best detection results compared to other models, while YOLOv4 performed the worst.

in efficiency and detection accuracy, achieving an mAP of 53.6% on VOC07+12 and 43.5% on COCO2017, with 64.0M Params and 142.2 GFLOPs. However, its performance is still limited compared to recent architectures. The state-of-the-art YOLOv9-E, with 70.0M Params and 245.5 GFLOPs, achieves an mAP of 76.0% on VOC07+12 and 55.6% on COCO2017, making it the most accurate model in this benchmark. The improvements stem from sophisticated network design and optimization techniques, enabling YOLOv9-E to excel across diverse object categories.

2) BALANCING ACCURACY AND COMPLEXITY

Larger models such as YOLOv5-X (with 86.3M Params and 204.2 GFLOPs) achieved impressive mAP values of 74.1% on VOC07+12 and 50.7% on COCO2017, demonstrating that increased complexity correlates with better accuracy. Similarly, YOLOv8-X, with 68.2M Params and 258.2 GFLOPs, achieved 73.4% on VOC07+12 and 53.9% on COCO2017, solidifying the trend that larger architectures tend to perform better. However, lightweight models like YOLOv5-S and YOLOv7-S cater to applications requiring faster inference and lower computational demand. YOLOv5-S, with just 7.0M Params and 15.9 GFLOPs, achieves a respectable mAP of 59.6% on VOC07+12 and 37.4% on COCO2017, making it ideal for resource-constrained environments. Similarly, YOLOv7-S, with 37.3M Params and 105.4 GFLOPs, achieves a 71.2% mAP on VOC07+12, just 3.8% lower than that of YOLOv9-E, making it a balanced choice between performance and efficiency. In comparison, older models like YOLOv4, despite having 64.0M Params

and 142.2 GFLOPs, achieved only a 53.6% mAP after prolonged training, highlighting their limitations compared to modern versions.

B. CATEGORY AND OBJECT SIZE ANALYSIS

The AP of objects across different categories and sizes provides valuable insights into the strengths and limitations of various YOLO models, guiding their applicability in specific use cases. The performance of the YOLO models on the VOC07+12 and COCO2017 datasets, as shown in Fig. 20, demonstrates their varying capabilities in detecting objects across diverse scenarios. On the VOC07+12 dataset, YOLOv9-E stands out as the top-performing model, achieving exceptional category-wise AP values. For example, YOLOv9-E achieved 82.4% AP for “aeroplane”, 86.1% AP for “cat”, and 64.4% AP for “bottle”, significantly outperforming YOLOv4, which only achieved 58.3%, 56.2%, and 43.8% AP for the same categories, respectively. In addition, YOLOv9-E has the highest detection accuracy for the most widely used objects in the transportation field, such as “bicycle”, “bus”, “car”, and “motorbike”. Therefore, it is more suitable for applications that require higher detection accuracy. This remarkable improvement is attributed to the advanced architecture and optimization techniques of YOLOv9-E, enabling it to detect subtle features and variations within object classes. YOLOv7-S also performed well on the VOC07+12 dataset, achieving an mAP of 71.2%, which is only 4.8% lower than YOLOv9-E. As for lightweight models, YOLOv7-S has the highest accuracy of over 90% on 20 target objects. This balance between accuracy

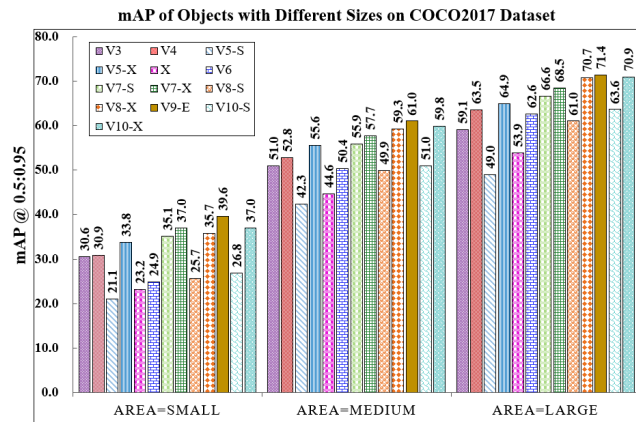


FIGURE 21. The mAP values of 80 objects divided into small, medium and large objects based on the COCO2017 dataset.

and computational efficiency makes YOLOv7-S suitable for applications that require reasonable detection performance without the computational demands of larger models. The analysis also highlights the limitations of older models like YOLOv4, which, despite having 64.0M Params, achieved relatively low AP values across most categories, demonstrating the need for more advanced architectures for modern object detection tasks. On the other hand, YOLOv10-S, with only 7.2M Params, offers competitive performance for lightweight deployments. In the detection accuracy of the two target categories, “cat” and “train”, YOLOv10-S exceeds YOLOv7-S with 82.3% and 77.9%, respectively. However, its lower AP values, such as a 57.0% AP for “bottle” and a 65.7% AP for “bird”, suggest it may struggle with complex or small objects. In all categories, the detection accuracy of the large model YOLOv9-E is the highest among all models. Therefore, if the detection accuracy is the primary consideration, YOLOv9-E is undoubtedly the best choice. It can achieve 74.8%, 85.4%, and 85.4% on objects with life characteristics such as “person”, “dog”, and “horse”, respectively. Some basic means of transportation such as “bicycle”, “bus”, and “motorbike” can reach 81.1%, 88.2%, and 77.6%, respectively. If the advantage of lightweight models in inference speed is taken into account, the model that can achieve the best detection accuracy and ensure speed is the YOLOv7-S model. YOLOv7-S achieves 69%, 77.8%, and 80.1% on “person”, “dog”, and “horse”, respectively. Means of transportation such as “bicycle”, “bus”, and “motorbike” can reach 77.3%, 86.1%, and 71.7%, respectively.

On the COCO2017 dataset, as shown in Fig. 21, which categorizes objects into small, medium, and large sizes, YOLOv9-E again leads with the highest mAP values across all size categories. It achieved an mAP of 39.6% for small objects, 61.0% for medium objects, and 71.4% for large objects. This consistent performance underscores the robustness of YOLOv9-E in detecting objects of varying sizes, making it ideal for applications requiring high accuracy across diverse scenarios, such as surveillance, traffic

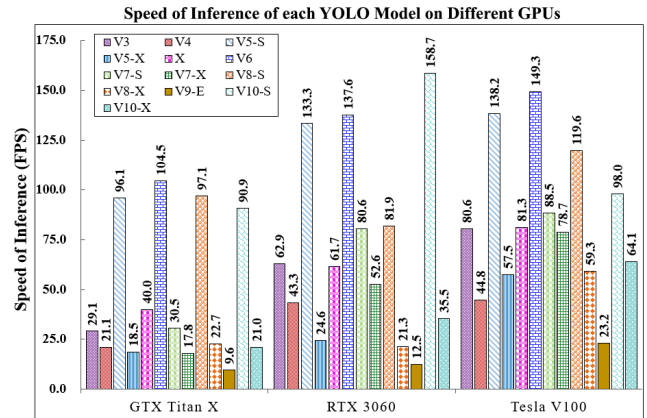


FIGURE 22. Comparison of inference speed in FPS across different YOLO versions running on various NVIDIA GPU models.

monitoring, and aerial imagery. In comparison, lightweight models like YOLOv5-S and YOLOv7-S performed adequately for medium and large objects but struggled with small object detection. For all YOLO versions of the lightweight model, YOLOv7-S performs best on the COCO2017 dataset. Other lightweight models, especially the latest YOLOv10-S, performs even worse on the COCO2017 dataset compared with YOLOv7-S, with no sign of overtaking. YOLOv5-S achieved an mAP of 21.1% for small objects and 49.0% for large objects, while YOLOv7-S achieved 35.1% and 66.6%, respectively. These results highlight the trade-offs of lightweight models: their smaller sizes and faster inference speeds make them suitable for resource-constrained environments, but they are less effective for detecting finer details or smaller objects. For instance, YOLOv5-S and YOLOv7-S are better suited for applications such as industrial automation and robotics, where the focus is on larger objects, and computational efficiency is critical. Among all models, YOLOv9-E has the highest detection accuracy for small, medium, and large objects. It can surpass the latest YOLOv10-X version which only achieved an mAP of 37.0% for small objects, 59.8% for medium objects, and 70.9% for large objects.

For applications requiring precise detection of small objects, such as pedestrian detection in dense urban environments or wildlife monitoring, larger models like YOLOv9-E or YOLOv8-X are more suitable. These models are capable of capturing fine details due to their higher parameter counts and advanced architectures, ensuring accurate detection even in challenging scenarios. YOLOv10-X, for instance, achieved an mAP of 59.8% on COCO2017 for medium-sized objects, showcasing its capability in detecting objects with moderate complexities. Overall, the choice of YOLO model depends on the specific application requirements such as (i) YOLOv9-E is ideal for high-stakes applications where accuracy across all object sizes is critical. It is particularly suited for scenarios requiring robust detection of small objects, such as aerial surveillance or fine-grained object classification. (ii) YOLOv7-S and YOLOv10-S are practical for

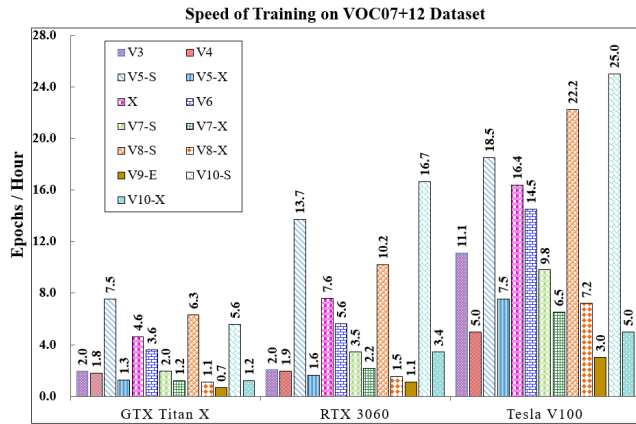


FIGURE 23. Speed of training in epochs per hour for different YOLO models using the VOC07+12 dataset across the GTX Titan X, RTX 3060, and Tesla V100 GPUs, demonstrating variations in training speed.

edge deployments and resource-limited systems, excelling in detecting medium to large objects while maintaining computational efficiency. (iii) Models like YOLOv8-X strike a balance, offering high accuracy for diverse object sizes with slightly reduced computational demands compared to YOLOv9-E. By aligning model selection with application-specific needs, practitioners can maximize performance and efficiency in deploying object detection systems. These results also emphasize the advancements in YOLO architectures, enabling better detection accuracy and flexibility across a wide range of use cases.

C. THE INFERENCE SPEED OF YOLO MODELS ACROSS GPUS AND THEIR PRACTICAL APPLICATIONS

The inference speed, quantified in FPS, is a pivotal metric for assessing the suitability of YOLO models for real-time applications. This metric determines how quickly a model can process individual frames, influencing its feasibility for tasks that demand immediate detection and response, such as autonomous driving, video surveillance, and robotics. Furthermore, the inference speed is a critical factor in resource-constrained environments like embedded systems, where efficiency and speed must align with limited computational resources. The performance of the YOLO models was benchmarked based on the COCO2017 dataset across three GPUs: GTX Titan X, RTX 3060, and Tesla V100, and the batch size we set for test is 1. The result is presented in Fig. 22. The analysis has reveals significant differences in inference speed, largely driven by architectural complexity, parameter count, and computational demands. The latest YOLOv10-S model has the fastest detection speed of all models on the RTX 3060, reaching 158.7 FPS. This result even exceeds its inference speed on the Tesla V100. The reason may be that the RTX 3060 is optimized for small-scale inference workloads, making it more suitable for the inference task of lightweight YOLO v10-S with a batch size of 1. The same model performs poorly when inferred on the Tesla V100 due to insufficient utilization of computing power under a small workload. Its highly optimized architecture minimizes

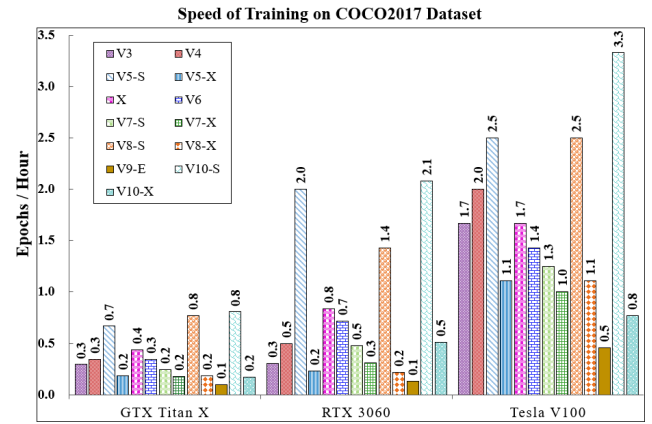


FIGURE 24. Comparison of training speeds, measured in epochs per hour, for various YOLO versions using the COCO2017 dataset across the GTX Titan X, RTX 3060, and Tesla V100 GPU.

computational overhead while maintaining competitive detection accuracy. This makes YOLOv10-S the ideal choice for real-time applications, such as traffic surveillance, industrial automation, and video analytics. By balancing speed and accuracy effectively, YOLOv10-S is particularly well-suited for tasks requiring high frame rates and immediate responses. Lightweight models like YOLOv5-S, YOLOv7-S, YOLOv8-S, and YOLOv10-S consistently demonstrated superior FPS compared to their larger counterparts, such as YOLOv5-X, YOLOv7-X, YOLOv8-X, YOLOv9-E, and YOLOv10-X, respectively. For example, YOLOv7-S outperformed YOLOv7-X in FPS across all GPUs, making it an excellent choice for applications where inference time is critical. The smaller parameter counts and reduced computational complexity of these “S” models render them ideal for deployment on edge devices and embedded systems. Scenarios such as real-time video analytics on drones, IoT devices, and mobile platforms can particularly benefit from the speed advantages offered by these lightweight versions.

Despite its state-of-the-art accuracy, YOLOv9-E recorded the lowest FPS across all GPUs due to its high parameter count (70.0M Params) and computational demands (245.5 GFLOPs). While this trade-off limits its applicability in real-time scenarios, YOLOv9-E is unmatched in precision-critical applications, such as medical imaging and high-resolution aerial analysis. For these tasks, where accuracy is paramount and time constraints are secondary, YOLOv9-E is the most suitable choice.

The choice of GPU significantly influenced inference speed across all models. The Tesla V100 consistently delivered the highest FPS, owing to its advanced architecture and superior computational capabilities. The RTX 3060 offered a compelling balance between cost and performance, achieving competitive FPS for most models, especially lightweight ones like YOLOv7-S. In contrast, the older GTX Titan X exhibited the slowest inference speeds across the board, making it less suitable for high-speed applications. The broader trends observed across YOLO versions emphasize the inherent

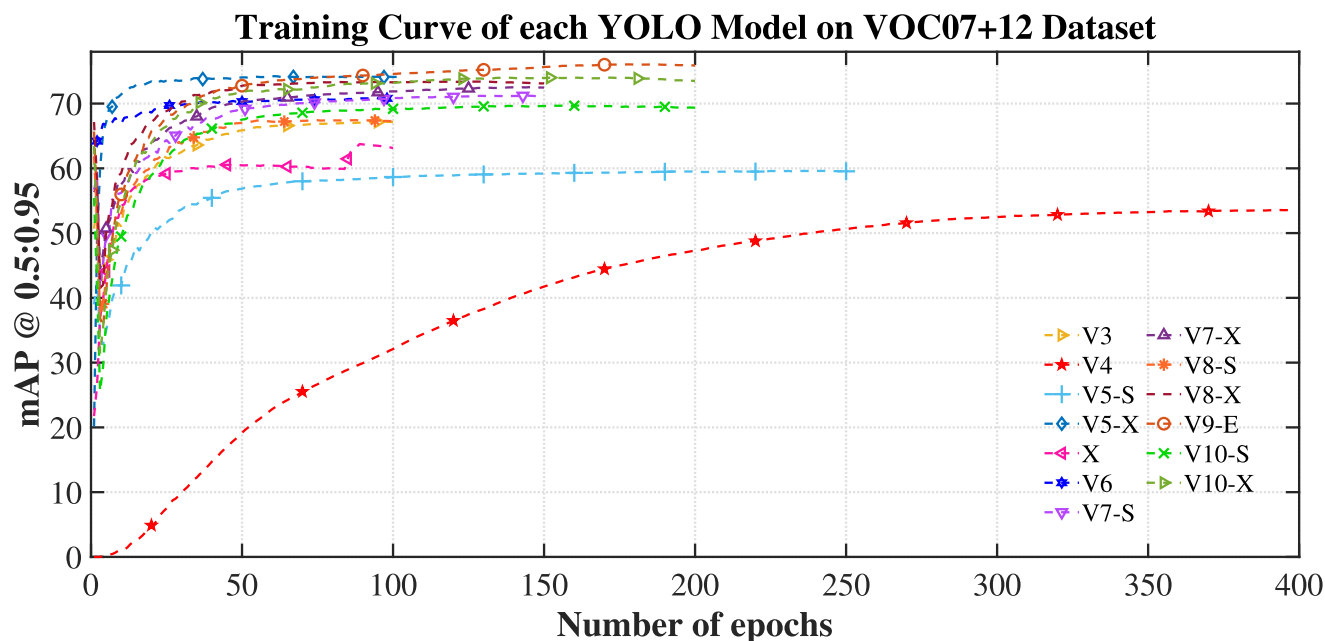


FIGURE 25. Convergence analysis of the YOLO models on the VOC07+12 dataset, depicting mAP (@0.5:0.95) as a function of number of training epochs. YOLOv5-X converges rapidly within 60 epochs, while YOLOv9-E requires about 200 epochs to reach its highest mAP. YOLOv4, exhibits significantly slower convergence, requiring approximately 400 epochs to achieve their peak mAP. Dashed lines with distinct symbols represent convergence data for each model version.

trade-offs between speed and accuracy. Larger models, such as YOLOv9-E and YOLOv5-X, excel in detection accuracy but incur substantial computational costs, leading to reduced FPS. Conversely, lightweight models prioritize speed and efficiency, making them more practical for real-time applications. Some practical applications are: (i) Real-Time Detection: YOLOv6 and lightweight “S” models (e.g., YOLOv5-S, YOLOv7-S) are highly recommended for real-time applications like traffic monitoring, robotics, and video surveillance, where speed is crucial. (ii) Resource-Constrained Deployments: Lightweight models are ideal for edge devices and embedded systems, enabling efficient operations without sacrificing much accuracy. (iii) High-Precision Tasks: YOLOv9-E is suited for applications demanding exceptional accuracy, such as medical diagnostics or satellite image analysis, where real-time processing is not a priority.

D. TRAINING SPEED AND RESOURCE OPTIMIZATION FOR YOLO MODELS

Deep learning models usually require plenty of time to complete the appropriate training epochs for the model to converge. Therefore, in our benchmark test, we also explored the training speed (epochs/hour) and the number of epochs required for each YOLO version to complete training using the two datasets VOC07+12 and COCO2017, on the three different GPU platforms. Based on these two parameters, we can also infer the total time required for each YOLO version to converge on the three different GPUs. The analysis of the training speed of each YOLO version

and their convergence can help YOLO users better choose the corresponding platform and plan the training strategy. On the VOC07+12 dataset, as shown in Fig. 23, the Tesla V100 demonstrated the fastest training speed across all YOLO models due to its advanced architecture and higher computational capacity. The training speeds of all YOLO versions on the Tesla V100 are faster than those on the other two GPUs. Their training speeds on the RTX 3060 are faster than those on the GTX Titan X. The speeds of “S” model of v5, v7, v8, and v10 are much faster than those of the “X” models. The training speeds of YOLOv9-E on the three GPUs are the slowest among all versions. The YOLOv5-S has the fastest training speed on the GTX Titan X, while the latest version YOLOv10-S has the quickest training speeds on the RTX 3060 and the Tesla V100 GPU. The RTX 3060 provided a middle ground in terms of training speed, offering better performance than the GTX Titan X but trailing behind the Tesla V100. Lightweight models such as YOLOv5-S and YOLOv8-S consistently outperformed their larger counterparts in terms of training speed. The slowest training speed is YOLOv9-E on the GTX Titan X at 0.7 epochs per hour, which is about four times slower than that on the Tesla V100 which can train about 3 epochs per hour.

The COCO2017 dataset, as shown in Fig. 24, being larger and more diverse, naturally demands longer training time and for comparison, the training speeds are measured in epochs per hour, for various YOLO versions on the COCO2017 dataset across the GTX Titan X, RTX 3060, and Tesla V100 GPUs. The training speed on the RTX 3060 is faster

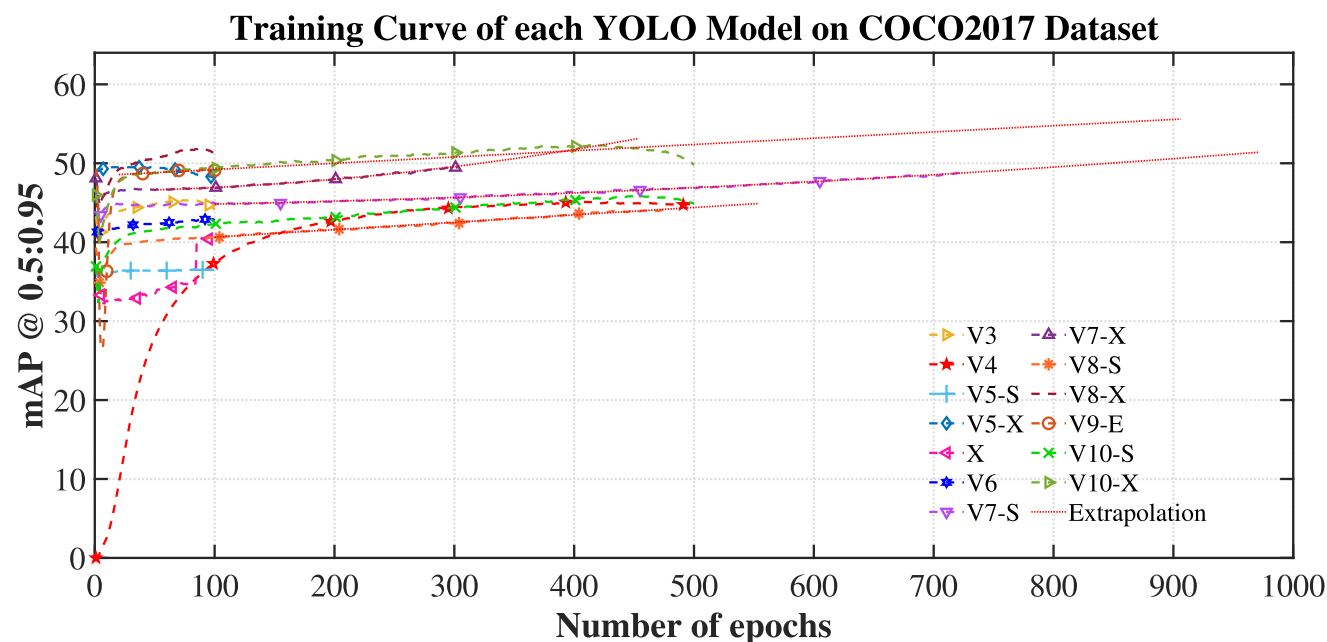


FIGURE 26. Convergence analysis of the YOLO models on the COCO2017 dataset, illustrating mAP (@0.5:0.95) progression across epochs. Models cannot reach convergence in an affordable time but be estimated by extrapolation, shown as red dotted lines, to reach optimal mAP values. Dashed lines with distinct symbols represent convergence data for each model version.

TABLE 7. Summary of the total time (hours) spent on training all YOLO models to get convergent on the VOC07+12 and COCO2017 datasets based on the GTX Titan X, RTX 3060, and Tesla V100 GPUs. *Total time inferred using an extrapolation method.

	GTX Titan X		RTX 3060		Tesla V100	
	VOC07+12	COCO2017	VOC07+12	COCO2017	VOC07+12	COCO2017
YOLOv3	49.0	278.8	47.0	270.6	8.6	49.2
YOLOv4	220.1	1,183.2	200.4	816.0	78.6	204
YOLOv5-S	31.2	124.5	146.4	41.5	12.0	33.2
YOLOv5-X	48.8	264.6	37.2	210.7	7.9	44.1
YOLOX	19.6	202.4	11.57	105.6	5.3	52.8
YOLOv6	26.3	272.6	16.9	131.6	6.58	65.8
YOLOv7-S	72.9	3,968.8*	41.5	2,032.8*	14.3	774.4*
YOLOv7-X	117.5	2,525.6*	66.7	1,443.2*	21.8	451.0*
YOLOv8-S	13.9	713.7*	8.7	384.3*	4.4	219.6*
YOLOv8-X	104.1	478.5	77.2	400.2	16.4	78.3
YOLOv9-E	254.5	9,331.8*	158.4	6,795*	58.7	1,986.6*
YOLOv10-S	27.5	537.6	9.7	224.0	6.4	134.4
YOLOv10-X	128.5	2,424.4	44.4	836.0	30.6	543.4

than that on the GTX Titan X. The speeds of “S” model of v5, v7, v8, and v10 are much faster than those of the “X” models. The training speed of YOLOv10-S on three GPUs is the fastest among all versions while YOLOv9-E has the slowest training speed on these three GPUs. YOLOv10-S can be trained at 0.8 epochs per hour, 2.1 epochs per hour,

and 3.3 epochs per hour on the three GPUs, respectively. The Tesla V100 GPU maintained its position as the fastest hardware for all models. For example, the training speed of YOLOv10-S is 3.3 epochs per hour on the Tesla V100, compared to 0.8 epochs per hour on the GTX Titan X. The slowest one is YOLOv9-E on the GTX Titan X at 0.1 epochs

per hour, which is about five times slower than that on the Tesla V100.

The convergence behaviors of YOLO models on two standard datasets, VOC07+12 and COCO2017, as shown in Fig. 25 and Fig. 26, respectively, provide critical insights into the computational resources and total time required for completing the training. On the VOC07+12 dataset, most models achieve convergence within 100 epochs. However, certain models, require extended training to reach their peak mAPs such as YOLOv9-E, achieving a 76.0% mAP after 177 epochs. Similarly, on the COCO2017 dataset, while many models converge within the initial 100 epochs, others necessitate over 500 epochs to reach their highest mAP values. The number of epochs required to achieve convergence varies significantly across models, reflecting the complexity of their architectures and the computational demands of their training processes. YOLOv5-X, for instance, achieves convergence on both datasets in approximately 60 epochs, making it one of the fastest and economic models in terms of training time. In contrast, older models like YOLOv4 require over 400 epochs to reach convergence on the VOC07+12 dataset, showcasing their slow convergence compared to modern YOLO versions. For YOLOv7-S, YOLOv7-X, YOLOv8-S, and YOLOv9-E, more training epochs are needed to reach convergence. Because the computational resources required for training are very huge, we use the extrapolation method. Based on the corresponding epochs we employed for models that had not converged within 100 or 500 epochs and the mAPs that can be achieved by convergence given on the official sites, we can predict the number of epochs required to reach convergence. For example, YOLOv7-S requires approximately 968 epochs of training to reach convergence, while the largest model YOLOv9-E requires a total of 906 epochs to reach convergence. This requires a lot of training time and computing resources. These estimates provide a practical basis for YOLO users to plan training sessions effectively, considering hardware constraints and budget limitations.

The total training time to convergence for each YOLO version using two datasets on three different GPUs is listed in Table 7. In general, the time required for training on VOC07+12 is less than 300 hours. The longest training time is for YOLOv9-E on the GTX Titan X, which takes 254.5 hours to get convergent using the VOC07+12 dataset. The shortest is YOLOv8-S on the Tesla V100, which only takes 4.4 hours, underscoring its efficiency in resource-constrained scenarios. In contrast, more complex models like YOLOv9-E and YOLOv7-X exhibited significantly longer training times due to their higher parameter counts and computational demands. For example, YOLOv9-E required 58.7 hours on the Tesla V100 and 254.5 hours on the GTX Titan X to achieve convergence on the VOC07+12 dataset. Training on the COCO2017 dataset takes longer than the VOC07+12 dataset. The fastest YOLO version is YOLOv5-S on the Tesla V100, which takes only 33.2 hours

for convergence. Models like YOLOv8-S continued to exhibit efficiency, requiring an extrapolated 219.6 hours on the V100, highlighting its applicability in scenarios where rapid training is prioritized. However, the predicted training time through extrapolation, for larger models like YOLOv9-E and YOLOv7-X revealed substantial resource demands. YOLOv9-E required 1,986.6 hours predicted through extrapolation on the Tesla V100 and a staggering amount of time, i.e., 9,331.8 hours on the GTX Titan X, illustrating its suitability for environments with abundant computational resources and no stringent time constraints. Understanding the convergence time of different YOLO versions is crucial for selecting the optimal model, especially when aiming to minimize training duration. For users considering renting GPU platforms, this information becomes even more valuable, as it enables the development of tailored training strategies that optimize resource allocation. By aligning the model choice with hardware capabilities and project requirements, users can effectively balance time, computational resources, and financial costs, ensuring a streamlined and cost-effective training process.

VI. CONCLUSION AND FUTURE CHALLENGES

This study provides a comprehensive evaluation of YOLO object detection models, highlighting their evolution, performance metrics, and practical applications. The benchmarks of the official versions of YOLO from v3 to v10 using the VOC07+12 and COCO2017 datasets across GTX Titan X, RTX 3060, and Tesla V100 GPUs have been conducted for analyzing detection accuracy (mAP), inference speed (FPS), training time, and computational complexity. The findings emphasize the trade-offs between accuracy, speed, and resource requirements, offering actionable guidance for model selection in diverse applications. State-of-the-art models like YOLOv9-E achieved exceptional detection accuracy, with an mAP of 76.0% on VOC07+12 and 55.6% on COCO2017, making them ideal for precision-critical tasks such as medical imaging and satellite analysis. Lightweight models, such as YOLOv5-S, YOLOv7-S, YOLOv8-S, and YOLOv10-S, demonstrated superior inference speed and shorter training times, making them optimal for real-time applications on resource-constrained devices. Among them, YOLOv7-S has the highest mAP value among these lightweight models. In addition, the results on training time show that YOLOv9-E takes the longest time to reach convergence among all versions, whether trained using the VOC07+12 or COCO2017 dataset. However, challenges persist in detecting small objects and maintaining robustness under adverse environmental conditions. Training analysis has revealed significant differences in convergence epochs and different models usually converge at different speeds. Some models require very few epochs and time to converge, while others require more time resources. These results underscore the importance of aligning model complexity with application-specific constraints, such as computational resources and deployment timelines. Future

development should address challenges in small object detection, real-time performance, and robustness across diverse conditions. Adaptive architectures and multimodal integration, combining visual, textual, and spatial data, represent promising research directions. Additionally, extending YOLO's capabilities into 3D object detection and keypoint localization, particularly for autonomous systems, remains a critical avenue for innovation. In conclusion, this study benchmarks YOLO's performance and provides strategic insights for its deployment, serving as a foundation for future research and application in both traditional and emerging domains.

ACKNOWLEDGMENT

The authors are grateful to the National Center for High-performance Computing (NCHC) in Taiwan for computer time and facilities.

REFERENCES

- [1] N. V. K. Medathati, H. Neumann, G. S. Masson, and P. Kornprobst, "Bio-inspired computer vision: Towards a synergistic approach of artificial and biological vision," *Comput. Vis. Image Understand.*, vol. 150, pp. 1–30, Sep. 2016, doi: [10.1016/j.cviu.2016.04.009](#).
- [2] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," *Psychol. Rev.*, vol. 65, no. 6, pp. 386–408, Nov. 1958, doi: [10.1037/h0042519](#).
- [3] J. Singh and R. Banerjee, "A study on single and multi-layer perceptron neural network," in *Proc. 3rd Int. Conf. Comput. Methodologies Commun. (ICCMC)*, Erode, India, Mar. 2019, pp. 35–40, doi: [10.1109/ICCMC.2019.8819775](#).
- [4] J. Canny, "A computational approach to edge detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vols. PAMI–8, no. 6, pp. 679–698, Nov. 1986, doi: [10.1109/TPAMI.1986.4767851](#).
- [5] J. Iivainen and A. J. E. Visa, "Shape recognition of irregular objects," *Proc. SPIE*, vol. 2904, pp. 25–32, Oct. 1996, doi: [10.1117/12.256280](#).
- [6] D. G. Lowe, "Robust model-based motion tracking through the integration of search and estimation," *Int. J. Comput. Vis.*, vol. 8, no. 2, pp. 113–122, Aug. 1992, doi: [10.1007/bf00127170](#).
- [7] R. Nevatia and K. Ramesh Babu, "Linear feature extraction and description," *Comput. Graph. Image Process.*, vol. 13, no. 3, pp. 257–269, Jul. 1980, doi: [10.1016/0146-664X\(80\)90049-0](#).
- [8] R. M. Haralick, "Using perspective transformations in scene analysis," *Comput. Graph. Image Process.*, vol. 13, no. 3, pp. 191–221, Jul. 1980, doi: [10.1016/0146-664X\(80\)90046-5](#).
- [9] M. I. Jordan and T. M. Mitchell, "Machine learning: Trends, perspectives, and prospects," *Science*, vol. 349, no. 6245, pp. 255–260, Jul. 2015, doi: [10.1126/science.aaa8415](#).
- [10] E. Akleman, "Deep learning," *Computer*, vol. 53, no. 9, pp. 1–23, Sep. 2020, doi: [10.1109/MC.2020.3004171](#).
- [11] W. S. Noble, "What is a support vector machine?" *Nature Biotechnol.*, vol. 24, no. 12, pp. 1565–1567, Dec. 2006, doi: [10.1038/nbt1206-1565](#).
- [12] T. K. Ho, "Random decision forests," in *Proc. 3rd Int. Conf. Document Anal. Recognit.*, vol. 1, Montreal, QC, Canada, Aug. 1995, pp. 278–282, doi: [10.1109/icdar.1995.598994](#).
- [13] J. Koushik, "Understanding convolutional neural networks," 2016, *arXiv:1605.09081*.
- [14] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998, doi: [10.1109/5.726791](#).
- [15] Z. Zou, K. Chen, Z. Shi, Y. Guo, and J. Ye, "Object detection in 20 years: A survey," *Proc. IEEE*, vol. 111, no. 3, pp. 257–276, Mar. 2023, doi: [10.1109/JPROC.2023.3238524](#).
- [16] R. Yang and Y. Yu, "Artificial convolutional neural network in object detection and semantic segmentation for medical imaging analysis," *Frontiers Oncol.*, vol. 11, Mar. 2021, Art. no. 638182, doi: [10.3389/fonc.2021.638182](#).
- [17] Y. Xiao, Z. Tian, J. Yu, Y. Zhang, S. Liu, S. Du, and X. Lan, "A review of object detection based on deep learning," *Multimedia Tools Appl.*, vol. 79, nos. 33–34, pp. 23729–23791, Jun. 2020, doi: [10.1007/s11042-020-08976-6](#).
- [18] A. Gupta, A. Anpalagan, L. Guan, and A. S. Khwaja, "Deep learning for object detection and scene perception in self-driving cars: Survey, challenges, and open issues," *Array*, vol. 10, Jul. 2021, Art. no. 100057, doi: [10.1016/j.array.2021.100057](#).
- [19] N. Surantha and W. R. Wicaksono, "Design of smart home security system using object recognition and PIR sensor," *Proc. Comput. Sci.*, vol. 135, pp. 465–472, Jan. 2018, doi: [10.1016/j.procs.2018.08.198](#).
- [20] Z. Li, M. Dong, S. Wen, X. Hu, P. Zhou, and Z. Zeng, "CLU-CNNs: Object detection for medical images," *Neurocomputing*, vol. 350, pp. 53–59, Jul. 2019, doi: [10.1016/j.neucom.2019.04.028](#).
- [21] G. Cheng and J. Han, "A survey on object detection in optical remote sensing images," *ISPRS J. Photogramm. Remote Sens.*, vol. 117, pp. 11–28, Jul. 2016, doi: [10.1016/j.isprsjprs.2016.03.014](#).
- [22] N. I. Glumov, E. I. Kolomiyetz, and V. V. Sergeyev, "Detection of objects on the image using a sliding window mode," *Opt. Laser Technol.*, vol. 27, no. 4, pp. 241–249, Aug. 1995, doi: [10.1016/0030-3992\(95\)93752-d](#).
- [23] P. C. Ng, "SIFT: Predicting amino acid changes that affect protein function," *Nucleic Acids Res.*, vol. 31, no. 13, pp. 3812–3814, Jul. 2003, doi: [10.1093/nar/gkg509](#).
- [24] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, vol. 1, San Diego, CA, USA, Jun. 2005, pp. 886–893, doi: [10.1109/CVPR.2005.177](#).
- [25] R. E. Schapire, "Explaining AdaBoost," in *Empirical Inference: Festschrift in Honor of Vladimir N. Vapnik*. Berlin, Germany: Springer, 2013, pp. 37–52.
- [26] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2014, pp. 580–587, doi: [10.1109/CVPR.2014.81](#).
- [27] K. He, X. Zhang, S. Ren, and J. Sun, "Spatial pyramid pooling in deep convolutional networks for visual recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 37, no. 9, pp. 1904–1916, Sep. 2015, doi: [10.1109/TPAMI.2015.2389824](#).
- [28] R. Girshick, "Fast R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Santiago, Chile, Dec. 2015, pp. 1440–1448, doi: [10.1109/ICCV.2015.169](#).
- [29] M. Everingham, S. M. A. Eslami, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The Pascal visual object classes challenge: A retrospective," *Int. J. Comput. Vis.*, vol. 111, no. 1, pp. 98–136, Jan. 2015, doi: [10.1007/s11263-014-0733-5](#).
- [30] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, Jun. 2017, doi: [10.1109/TPAMI.2016.2577031](#).
- [31] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 2, pp. 386–397, Feb. 2020, doi: [10.1109/TPAMI.2018.2844175](#).
- [32] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 770–778, doi: [10.1109/CVPR.2016.90](#).
- [33] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.
- [34] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 779–788, doi: [10.1109/CVPR.2016.91](#).
- [35] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot MultiBox detector," in *Proc. 14th Eur. Conf. Comput. Vis.*, Amsterdam, Netherlands, Jan. 2016, pp. 21–37, doi: [10.1007/978-3-319-46448-0_2](#).
- [36] A. Veit, T. Matera, L. Neumann, J. Matas, and S. Belongie, "COCO-text: Dataset and benchmark for text detection and recognition in natural images," 2016, *arXiv:1601.07140*.
- [37] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 2, pp. 318–327, Feb. 2020, doi: [10.1109/TPAMI.2018.2858826](#).

- [38] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Honolulu, HI, USA, Jul. 2017, pp. 936–944, doi: [10.1109/CVPR.2017.106](https://doi.org/10.1109/CVPR.2017.106).
- [39] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," 2018, *arXiv:1804.02767*.
- [40] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, "YOLOv9: Learning what you want to learn using programmable gradient information," in *Proc. 18th Eur. Conf. Comput. Vis.*, Milan, Italy, Oct. 2024, pp. 1–21, doi: [10.1007/978-3-031-72751-1_1](https://doi.org/10.1007/978-3-031-72751-1_1).
- [41] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "YOLOv10: Real-time end-to-end object detection," 2024, *arXiv:2405.14458*.
- [42] P. Jiang, D. Ergu, F. Liu, Y. Cai, and B. Ma, "A review of YOLO algorithm developments," *Proc. Comput. Sci.*, vol. 199, pp. 1066–1073, Jan. 2022, doi: [10.1016/j.procs.2022.01.135](https://doi.org/10.1016/j.procs.2022.01.135).
- [43] Z.-Q. Zhao, P. Zheng, S.-T. Xu, and X. Wu, "Object detection with deep learning: A review," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 11, pp. 3212–3232, Nov. 2019, doi: [10.1109/TNNLS.2018.2876865](https://doi.org/10.1109/TNNLS.2018.2876865).
- [44] U. Sirisha, S. P. Praveen, P. N. Srinivasu, P. Barsocchi, and A. K. Bhoi, "Statistical analysis of design aspects of various YOLO-based deep learning models for object detection," *Int. J. Comput. Intell. Syst.*, vol. 16, no. 1, Aug. 2023, Art. no. 126, doi: [10.1007/s44196-023-00302-w](https://doi.org/10.1007/s44196-023-00302-w).
- [45] J. Terven, D.-M. Córdova-Esparza, and J.-A. Romero-González, "A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS," *Mach. Learn. Knowl. Extrac.*, vol. 5, no. 4, pp. 1680–1716, Nov. 2023, doi: [10.3390/make5040083](https://doi.org/10.3390/make5040083).
- [46] M. Hussain, "YOLO-v1 to YOLO-v8, the rise of YOLO and its complementary nature toward digital manufacturing and industrial defect detection," *Machines*, vol. 11, no. 7, p. 677, Jun. 2023, doi: [10.3390/machines11070677](https://doi.org/10.3390/machines11070677).
- [47] M. Hussain, "YOLOv1 to v8: Unveiling each variant—A comprehensive review of YOLO," *IEEE Access*, vol. 12, pp. 42816–42833, 2024, doi: [10.1109/ACCESS.2024.3378568](https://doi.org/10.1109/ACCESS.2024.3378568).
- [48] A. Neubeck and L. Van Gool, "Efficient non-maximum suppression," in *Proc. 18th Int. Conf. Pattern Recognit. (ICPR)*, vol. 3, Aug. 2006, pp. 850–855, doi: [10.1109/ICPR.2006.479](https://doi.org/10.1109/ICPR.2006.479).
- [49] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Boston, MA, USA, Jun. 2015, pp. 1–9, doi: [10.1109/CVPR.2015.7298594](https://doi.org/10.1109/CVPR.2015.7298594).
- [50] J. Redmon and A. Farhadi, "YOLO9000: Better, faster, stronger," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Honolulu, HI, USA, Jul. 2017, pp. 6517–6525, doi: [10.1109/CVPR.2017.690](https://doi.org/10.1109/CVPR.2017.690).
- [51] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Miami, FL, USA, Jun. 2009, pp. 248–255, doi: [10.1109/CVPR.2009.5206848](https://doi.org/10.1109/CVPR.2009.5206848).
- [52] A. Likas, N. Vlassis, and J. J. Verbeek, "The global k-means clustering algorithm," *Pattern Recognit.*, vol. 36, no. 2, pp. 451–461, Feb. 2003, doi: [10.1016/S0031-3203\(02\)00060-2](https://doi.org/10.1016/S0031-3203(02)00060-2).
- [53] A. Bochkovskiy, C.-Y. Wang, and H.-Y. Mark Liao, "YOLOv4: Optimal speed and accuracy of object detection," 2020, *arXiv:2004.10934*.
- [54] C.-Y. Wang, H.-Y. Mark Liao, Y.-H. Wu, P.-Y. Chen, J.-W. Hsieh, and I.-H. Yeh, "CSPNet: A new backbone that can enhance learning capability of CNN," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Seattle, WA, USA, Jun. 2020, pp. 1571–1580, doi: [10.1109/CVPRW50498.2020.00203](https://doi.org/10.1109/CVPRW50498.2020.00203).
- [55] D. Misra, "Mish: A self regularized non-monotonic activation function," in *Proc. Brit. Mach. Vis. Conf.*, 2020, pp. 1–14.
- [56] Z. Zheng, P. Wang, D. Ren, W. Liu, R. Ye, Q. Hu, and W. Zuo, "Enhancing geometric factors in model learning and inference for object detection and instance segmentation," *IEEE Trans. Cybern.*, vol. 52, no. 8, pp. 8574–8586, Aug. 2022, doi: [10.1109/TCYB.2021.3095305](https://doi.org/10.1109/TCYB.2021.3095305).
- [57] J. Xu, Z. Li, B. Du, M. Zhang, and J. Liu, "Reluplex made more practical: Leaky ReLU," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Rennes, France, Jul. 2020, pp. 1–7, doi: [10.1109/ISCC50000.2020.9219587](https://doi.org/10.1109/ISCC50000.2020.9219587).
- [58] X. Glorot, A. Bordes, and Y. Bengio, "Deep sparse rectifier neural networks," in *Proc. 14th Int. Conf. Artif. Intell. Statist. (AISTATS)*, Fort Lauderdale, FL, USA, Apr. 2011, pp. 315–323.
- [59] P. Ramachandran, B. Zoph, and Q. V. Le, "Searching for activation functions," 2017, *arXiv:1710.05941*.
- [60] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, "Distance-IoU loss: Faster and better learning for bounding box regression," in *Proc. AAAI Conf. Artif. Intell.*, Apr. 2020, vol. 34, no. 7, pp. 12993–13000, doi: [10.1609/aaai.v34i07.6999](https://doi.org/10.1609/aaai.v34i07.6999).
- [61] J. Yao, J. Qi, J. Zhang, H. Shao, J. Yang, and X. Li, "A real-time detection algorithm for kiwifruit defects based on YOLOv5," *Electronics*, vol. 10, no. 14, p. 1711, Jul. 2021, doi: [10.3390/electronics10141711](https://doi.org/10.3390/electronics10141711).
- [62] Z. Ge, S. Liu, F. Wang, Z. Li, and J. Sun, "YOLOX: Exceeding YOLO series in 2021," 2021, *arXiv:2107.08430*.
- [63] M. Tan, R. Pang, and Q. V. Le, "EfficientDet: Scalable and efficient object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Seattle, WA, USA, Jun. 2020, pp. 10778–10787, doi: [10.1109/CVPR42600.2020.01079](https://doi.org/10.1109/CVPR42600.2020.01079).
- [64] H. Ouyang, "DEYO: DETR with YOLO for step-by-step object detection," 2022, *arXiv:2211.06588*.
- [65] C. Li, L. Li, H. Jiang, K. Weng, Y. Geng, L. Li, Z. Ke, Q. Li, M. Cheng, W. Nie, Y. Li, B. Zhang, Y. Liang, L. Zhou, X. Xu, X. Chu, X. Wei, and X. Wei, "YOLOv6: A single-stage object detection framework for industrial applications," 2022, *arXiv:2209.02976*.
- [66] C.-Y. Wang, A. Bochkovskiy, and H.-Y. Mark Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 7464–7475, doi: [10.1109/CVPR52729.2023.00721](https://doi.org/10.1109/CVPR52729.2023.00721).
- [67] Y. Lee, J.-W. Hwang, S. Lee, Y. Bae, and J. Park, "An energy and GPU-computation efficient backbone network for real-time object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Long Beach, CA, USA, Jun. 2019, pp. 752–760, doi: [10.1109/CVPRW.2019.00103](https://doi.org/10.1109/CVPRW.2019.00103).
- [68] I. Lazarevich, M. Grimaldi, R. Kumar, S. Mitra, S. Khan, and S. Sah, "YOLOBench: Benchmarking efficient object detectors on embedded systems," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshops (ICCVW)*, Oct. 2023, pp. 1161–1170, doi: [10.1109/ICCVW60793.2023.00126](https://doi.org/10.1109/ICCVW60793.2023.00126).
- [69] Y. Luo, Y. Ci, S. Jiang, and X. Wei, "A novel lightweight real-time traffic sign detection method based on an embedded device and YOLOv8," *J. Real-Time Image Process.*, vol. 21, no. 2, Jan. 2024, Art. no. 24, doi: [10.1007/s11554-023-01403-7](https://doi.org/10.1007/s11554-023-01403-7).
- [70] Y. Cai, Y. Zhou, Q. Han, J. Sun, X. Kong, J. Li, and X. Zhang, "Reversible column networks," 2022, *arXiv:2212.11696*.
- [71] H. Bao, L. Dong, S. Piao, and F. Wei, "BEiT: BERT pre-training of image transformers," 2021, *arXiv:2106.08254*.
- [72] C. Lee, S. Xie, P. W. Gallagher, Z. Zhang, and Z. Tu, "Deeply-supervised nets," in *Proc. 18th Int. Conf. Artif. Intell. Statist. (AISTATS)*, San Diego, CA, USA, Jan. 2014, pp. 562–570.
- [73] W. Lan, J. Dang, Y. Wang, and S. Wang, "Pedestrian detection based on YOLO network model," in *Proc. IEEE Int. Conf. Mechatronics Autom. (ICMA)*, Changchun, China, Aug. 2018, pp. 1547–1551, doi: [10.1109/ICMA.2018.8484698](https://doi.org/10.1109/ICMA.2018.8484698).
- [74] M. V. Priya, and D. S. Pankaj, "3DYOLO: Real-time 3D object detection in 3D point clouds for autonomous driving," in *Proc. IEEE Int. India Geosci. Remote Sens. Symp. (InGARSS)*, Ahmedabad, India, Dec. 2021, pp. 41–44, doi: [10.1109/InGARSS51564.2021.9791912](https://doi.org/10.1109/InGARSS51564.2021.9791912).
- [75] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Providence, RI, USA, Jun. 2012, pp. 3354–3361, doi: [10.1109/CVPR.2012.6248074](https://doi.org/10.1109/CVPR.2012.6248074).
- [76] C. Dewi, R.-C. Chen, X. Jiang, and H. Yu, "Deep convolutional neural network for enhancing traffic sign recognition developed on YOLO V4," *Multimedia Tools Appl.*, vol. 81, no. 26, pp. 37821–37845, Apr. 2022, doi: [10.1007/s11042-022-12962-5](https://doi.org/10.1007/s11042-022-12962-5).
- [77] M. O. Lawal, "Tomato detection based on modified YOLOv3 framework," *Sci. Rep.*, vol. 11, no. 1, Jan. 2021, Art. no. 1447, doi: [10.1038/s41598-021-81216-5](https://doi.org/10.1038/s41598-021-81216-5).
- [78] H. Pang, Y. Zhang, W. Cai, B. Li, and R. Song, "A real-time object detection model for orchard pests based on improved YOLOv4 algorithm," *Sci. Rep.*, vol. 12, no. 1, Aug. 2022, Art. no. 13557, doi: [10.1038/s41598-022-17826-4](https://doi.org/10.1038/s41598-022-17826-4).
- [79] Z. Qin, W. Wang, K.-H. Dammer, L. Guo, and Z. Cao, "Ag-YOLO: A real-time low-cost detector for precise spraying with case study of palms," *Frontiers Plant Sci.*, vol. 12, Dec. 2021, Art. no. 753603, doi: [10.3389/fpls.2021.753603](https://doi.org/10.3389/fpls.2021.753603).

- [80] Q. Li, J. Song, J. Ning, and J. Yuan, "The detailed data on the neural compute stick acceleration performance," in *Proc. Chin. Autom. Congr. (CAC)*, Hangzhou, China, Nov. 2019, pp. 4959–4962, doi: [10.1109/CAC48633.2019.8996841](https://doi.org/10.1109/CAC48633.2019.8996841).
- [81] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "Squeezenet: AlexNet-level accuracy with 50x fewer parameters and <0.5 MB model size," 2016, *arXiv:1602.07360*.
- [82] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Salt Lake City, UT, USA, Jun. 2018, pp. 4510–4520, doi: [10.1109/CVPR.2018.00474](https://doi.org/10.1109/CVPR.2018.00474).
- [83] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "ShuffleNet v2: Practical guidelines for efficient CNN architecture design," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Munich, Germany, Jan. 2018, pp. 122–138, doi: [10.1007/978-3-030-01264-9_8](https://doi.org/10.1007/978-3-030-01264-9_8).
- [84] S. Ramachandran, J. George, S. Skaria, and V. V. V., "Using YOLO based deep learning network for real time detection and localization of lung nodules from low dose CT scans," *Proc. SPIE*, vol. 10575, p. 53, Feb. 2018, doi: [10.1117/12.2293699](https://doi.org/10.1117/12.2293699).
- [85] S. G. Armato et al., "The lung image database consortium (LIDC) and image database resource initiative (IDRI): A completed reference database of lung nodules on CT scans," *Med. Phys.*, vol. 38, no. 2, pp. 915–931, Jan. 2011, doi: [10.1118/1.3528204](https://doi.org/10.1118/1.3528204).
- [86] M. Hammami, D. Friboulet, and R. Kechichian, "Cycle GAN-based data augmentation for multi-organ detection in CT images via YOLO," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Abu Dhabi, United Arab Emirates, Oct. 2020, pp. 390–393, doi: [10.1109/ICIP40778.2020.9191127](https://doi.org/10.1109/ICIP40778.2020.9191127).
- [87] C. Chu, A. Zhmoginov, and M. Sandler, "CycleGAN, a master of steganography," 2017, *arXiv:1712.02950*.
- [88] O. Jimenez-del-Toro, H. Müller, M. Krenn, K. Gruenberg, A. A. Taha, M. Winterstein, I. Eggel, A. Foncubierta-Rodríguez, O. Goksel, A. Jakab, and G. Kontokotsios, "Cloud-based evaluation of anatomical structure segmentation and landmark detection algorithms: VISCERAL anatomy benchmarks," *IEEE Trans. Med. Imag.*, vol. 35, no. 11, pp. 2459–2475, Nov. 2016, doi: [10.1109/TMI.2016.2578680](https://doi.org/10.1109/TMI.2016.2578680).
- [89] V. Alamelu and S. Thilagamani, "Lion based butterfly optimization with improved YOLO-v4 for heart disease prediction using IoMT," *Inf. Technol. Control*, vol. 51, no. 4, pp. 692–703, Dec. 2022, doi: [10.5755/j01.ite.51.4.31323](https://doi.org/10.5755/j01.ite.51.4.31323).
- [90] N. Singh Pun, S. Kumar Sonbhadra, S. Agarwal, and G. Rai, "Monitoring COVID-19 social distancing with person detection and tracking via fine-tuned YOLO v3 and deepsort techniques," 2020, *arXiv:2005.01385*.
- [91] C. Guo, X.-L. Lv, Y. Zhang, and M.-L. Zhang, "Improved YOLOv4-tiny network for real-time electronic component detection," *Sci. Rep.*, vol. 11, no. 1, Nov. 2021, Art. no. 22744, doi: [10.1038/s41598-021-02225-y](https://doi.org/10.1038/s41598-021-02225-y).
- [92] J. Fu, J. Liu, H. Tian, Y. Li, Y. Bao, Z. Fang, and H. Lu, "Dual attention network for scene segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Long Beach, CA, USA, Jun. 2019, pp. 3141–3149, doi: [10.1109/CVPR.2019.00326](https://doi.org/10.1109/CVPR.2019.00326).
- [93] Y. Liang, G. Qin, M. Sun, J. Yan, and H. Jiang, "MAFNet: Multi-style attention fusion network for salient object detection," *Neurocomputing*, vol. 422, pp. 22–33, Jan. 2021, doi: [10.1016/j.neucom.2020.09.033](https://doi.org/10.1016/j.neucom.2020.09.033).
- [94] M. Zhang and L. Yin, "Solar cell surface defect detection based on improved YOLO v5," *IEEE Access*, vol. 10, pp. 80804–80815, 2022, doi: [10.1109/ACCESS.2022.3195901](https://doi.org/10.1109/ACCESS.2022.3195901).
- [95] Q. Wang, B. Wu, P. Zhu, P. Li, W. Zuo, and Q. Hu, "ECA-net: Efficient channel attention for deep convolutional neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Seattle, WA, USA, Jun. 2020, pp. 11531–11539, doi: [10.1109/CVPR42600.2020.01155](https://doi.org/10.1109/CVPR42600.2020.01155).
- [96] T.-T.-H. Vu, D.-L. Pham, and T.-W. Chang, "A YOLO-based real-time packaging defect detection system," *Proc. Comput. Sci.*, vol. 217, pp. 886–894, Jan. 2023, doi: [10.1016/j.procs.2022.12.285](https://doi.org/10.1016/j.procs.2022.12.285).
- [97] P. Shi, Q. Jiang, C. Shi, J. Xi, G. Tao, S. Zhang, Z. Zhang, B. Liu, X. Gao, and Q. Wu, "Oil well detection via large-scale and high-resolution remote sensing images based on improved YOLO v4," *Remote Sens.*, vol. 13, no. 16, p. 3243, Aug. 2021, doi: [10.3390/rs13163243](https://doi.org/10.3390/rs13163243).
- [98] B. V. B. Prabhu, R. Lakshmi, R. Ankitha, M. S. Prateeksha, and N. C. Priya, "RescueNet: YOLO-based object detection model for detection and counting of flood survivors," *Model. Earth Syst. Environ.*, vol. 8, no. 4, pp. 4509–4516, May 2022, doi: [10.1007/s40808-022-01414-6](https://doi.org/10.1007/s40808-022-01414-6).
- [99] J. Gao, Y. Chen, Y. Wei, and J. Li, "Detection of specific building in remote sensing images using a novel YOLO-S-CIOU model. Case: Gas station identification," *Sensors*, vol. 21, no. 4, p. 1375, Feb. 2021, doi: [10.3390/s21041375](https://doi.org/10.3390/s21041375).
- [100] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Salt Lake City, UT, USA, Jun. 2018, pp. 7132–7141, doi: [10.1109/CVPR.2018.00745](https://doi.org/10.1109/CVPR.2018.00745).
- [101] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, May 2017, doi: [10.1145/3065386](https://doi.org/10.1145/3065386).
- [102] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, and Y. LeCun, "OverFeat: Integrated recognition, localization and detection using convolutional networks," 2013, *arXiv:1312.6229*.
- [103] Y. Qing, W. Liu, L. Feng, and W. Gao, "Improved YOLO network for free-angle remote sensing target detection," *Remote Sens.*, vol. 13, no. 11, p. 2171, Jun. 2021, doi: [10.3390/rs13112171](https://doi.org/10.3390/rs13112171).
- [104] M. Masoom S., Q. Zhang, P. Dai, Y. Jia, Y. Zhang, J. Zhu, and J. Wang, "Early smoke detection based on improved YOLO-PCA network," *Fire*, vol. 5, no. 2, p. 40, Mar. 2022, doi: [10.3390/fire5020040](https://doi.org/10.3390/fire5020040).
- [105] W.-L. Mao, W.-C. Chen, H. I. K. Fathurrahman, and Y.-H. Lin, "Deep learning networks for real-time regional domestic waste detection," *J. Cleaner Prod.*, vol. 344, Apr. 2022, Art. no. 131096, doi: [10.1016/j.jclepro.2022.131096](https://doi.org/10.1016/j.jclepro.2022.131096).
- [106] J. Pak, H. Jeong, S. Lee, W. Lee, J. Kim, and J. Chung, "Performance of GTX Titan X GPUs and code optimization," in *Proc. 33rd Int. Symp. Lattice Field Theory*, Kobe, Japan, Jul. 2016, pp. 1–7, doi: [10.22323/1.251.0038](https://doi.org/10.22323/1.251.0038).
- [107] M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Zurich, Switzerland, Jan. 2014, pp. 818–833, doi: [10.1007/978-3-319-10590-1_53](https://doi.org/10.1007/978-3-319-10590-1_53).
- [108] NVIDIA Corporation, "NVIDIA Tesla V100 GPU architecture," NVIDIA, Tech. Rep., Aug. 2017. [Online]. Available: <https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf>



ZHENGMAO HUA received the M.S. degree in electrical engineering from Hanyang University, Seoul, South Korea, in 2025.

His research interests include computer vision and object detection.



KAVIYA ARANGANADIN (Member, IEEE) received the B.Tech. degree in electrical and electronic engineering from SMVEC, Pondicherry University, Madagadipet, India, in 2016. She is currently pursuing the Ph.D. degree with the Multidisciplinary Computational Laboratory, Department of Electrical and Biomedical Engineering, Hanyang University, Seoul, South Korea.

Her research interests include particle-in-cell and fluid simulations of plasma and magnetron, electric circuit analysis, and finite-element simulations of microwave devices. She is also involved in computational materials research using density functional theory, molecular dynamics, and machine learning, and in the development of artificial intelligence-driven technologies, such as automatic speech recognition and computer vision.



CHENG-CHENG YEH is currently pursuing the Ph.D. degree with National Cheng Kung University (NCKU), Taiwan.

His research interests include artificial intelligence (AI), high-performance computing (HPC), and radiotherapy. He has more than 15 years of experience in the international IT industry, working at HP, SGI, Unisys, Cray Inc., etc., advancing to regional leadership in APAC. He later transitioned into entrepreneurship, focusing on bioinformatics, HPC, and internet medical applications, leveraging his expertise in AI, HPC, and cloud technologies. In 2016, he expanded his business into China, successfully launched a mini-supercomputer, and received the “Talent’s Green Card of Guangzhou Municipality” for his achievements in technological innovation and business development. He is engaged in academic research, collaborating with NCKU, National Taiwan University, and Hanyang University, South Korea, on genomic sequencing, internet of things, blockchain, collaborative intelligent transportation system, high definition mapping, and AI. In recent years, he has strategically aimed at AI, conducting image recognition, automatic speech recognition (ASR), large language models, and retrieval augmented generation.



XINHE HAI received the M.S. degree in electrical engineering from Hanyang University, Seoul, South Korea, in 2025.

Her research interests include multilingual ASR for medical service and emergency assistance systems.



CHEN-YUN HUANG received the Master of Engineering degree in computer science from the College of Electrical Engineering, National Chiao Tung University (NCTU), Taiwan, in 2005.

Her research focused on information security technology, software engineering, and system development. After working in the information technology industry, she became deeply involved in HPC and related technologies, including performance optimization and distributed computing architecture. She worked for some startup companies to be the leader of software engineering and system product development.



TSAN-CHUEN LEUNG received the Ph.D. degree in physics from Iowa State University, USA, in 1991.

He is currently a Professor with the Department of Physics, National Chung Cheng University, Chiayi, Taiwan. His research interests include first-principles calculations, computational physics, and condensed theoretical physics, including studies of metal surfaces, magnetic thin films, nanostructures, phonon dispersion in conventional and nanomaterials, and the electronic and magnetic properties of transition metal oxides.



HUA-YI HSU received the Ph.D. degree in mechanical engineering from Northwestern University, Evanston, IL, USA, in 2008.

She is currently a Full Professor with the Department of Mechanical Engineering, National Taipei University of Technology, Taipei, Taiwan. Her research interests include theoretical and computational fluid dynamics, phase change and heat transfer, boiling, condensation, surface effect, particle-in-cell simulations of plasmas, microwave tubes, electron emissions, and machine learning.



YUNG-CHIANG LAN received the Ph.D. degree in engineering and systems science from National Tsing Hua University, Taiwan, in 2002.

He joined the Department of Photonics, National Cheng Kung University (NCKU), Taiwan, as an Assistant Professor, in 2004, became an Associate Professor, in 2007, and has been a Full Professor, since 2011. His research interests include the simulation and analysis of optical properties of metamaterials, plasmonic photonic devices, and transformation optics. His recent work focuses on surface plasmons, nanostructures, and electron beam-plasmon interactions, integrating advanced simulation techniques for novel optical phenomena, and deep learning.



MING-CHIEH LIN (Senior Member, IEEE) received the B.S., M.S., and Ph.D. degrees in electrophysics from National Chiao Tung University, Hsinchu, Taiwan, in 1996, 1998, and 2002, respectively.

He is currently a Full Professor with the Department of Electrical and Biomedical Engineering, Hanyang University, Seoul, South Korea, where he is involved in particle-in-cell and fluid simulations of plasmas, microwave tubes, electron emissions, and terahertz-wave sources, and first-principles computations of surface physics and heavy ion therapy. He is also the Founder of the Multidisciplinary Computational Laboratory, Hanyang University, where he focuses on HPC applications, combining computational electromagnetics and quantum mechanics for studying complex systems and biomedical applications, and providing professional training and consulting services based on more than 20 years of academic and industrial experience in related fields.

...